République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي و البحث العلمي

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)

École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Mémoire de fin d'études

Pour l'obtention du diplôme d'Ingénieur d'Etat en Informatique

Option : Systèmes Informatiques et Logiciels

Contextualisation des Dépendances Logicielles Vulnérables pour la Sécurisation des Applications Java

Réalisé par :
M. Belhout Ahmed

Encadré par :
Dr. YESSAD Lamia (ESI)
Dr. SERIAI Abdelhak-Djamel (LIRMM)

Promotion : 2023/2024

Dédicace

“

À ma famille, pour leur soutien constant et leur encouragement. Votre présence et votre aide m'ont donné la force et l'inspiration nécessaires. Je vous remercie du fond du cœur pour tout ce que vous avez fait pour moi.

À mes amis et collègues, pour les moments partagés et leur soutien précieux. Votre présence a rendu ce chemin plus agréable et vos encouragements ont été essentiels. Merci pour tout.

Merci.

”

- Ahmed

Remerciements

C'est avec une profonde gratitude que je consacre cette page à tous ceux qui ont soutenu, de près ou de loin, la réalisation de ce travail.

Je tiens d'abord à exprimer ma gratitude envers Dieu, Allah, pour m'avoir doté de la clarté d'esprit, de la force et de la détermination nécessaires pour mener à bien ce projet.

Ensuite, j'exprime ma sincère gratitude à **Dr. Abdelhak-Djamel SERIAI**, Maître de conférences (HDR) à l'université de Montpellier, pour m'avoir donné l'opportunité d'explorer ce sujet motivant au cours de mon stage.

Je tiens à exprimer ma profonde reconnaissance à mon encadrante, **Dr Lamia YESSAD**, Maître de conférences B à l'ESI, pour son accompagnement constant et ses conseils précieux qui ont guidé mon travail.

Je souhaite également remercier ma famille pour leur soutien inconditionnel, leur patience et leur compréhension. Leur présence et leurs encouragements ont été essentiels pour mon engagement et la réussite de ce projet.

Enfin, je souhaite exprimer mes sincères remerciements aux professeurs de l'ESI Alger. Leur enseignement m'a fourni des bases solides dans divers domaines, ce qui a été essentiel pour la réalisation de ce stage.

Résumé

Le développement d'applications contemporaines nécessite fréquemment l'intégration de diverses dépendances, qu'elles soient open source ou propriétaires. Ces composants tiers, tels que les bibliothèques, les frameworks et les modules, sont intégrés pour accélérer le processus de développement, ajouter des fonctionnalités avancées et éviter de développer des solutions "from scratch" (à partir de zéro).

Cependant, ces dépendances peuvent présenter des vulnérabilités, qu'elles soient déjà connues ou non, qui pourraient être exploitées par des cyberattaquants. De telles failles peuvent mettre en péril la sécurité de l'application, compromettre des données sensibles, perturber son fonctionnement normal, voire permettre un contrôle complet du système sur lequel elle fonctionne.

L'objectif de ce travail est d'explorer les mécanismes de contextualisation des dépendances logicielles vulnérables. Cette contextualisation permet aux développeurs de comprendre en profondeur les risques spécifiques associés à chaque dépendance. Elle facilite également la priorisation des actions de correction des vulnérabilités détectées en mettant l'accent sur celles qui présentent le plus grand danger pour l'application.

Ce travail vise à examiner les méthodes actuelles de contextualisation des dépendances logicielles vulnérables dans une application Java à analyser, en mettant en lumière les lacunes des méthodes existantes et en cherchant à contribuer à l'amélioration de la contextualisation des dépendances vulnérables grâce à l'utilisation de techniques d'analyse du code source.

La solution proposée a été soumise à plusieurs types de tests afin de garantir sa robustesse et son efficacité. Ces tests ont validé la capacité de l'approche à contextualiser avec précision les dépendances vulnérables, tout en tenant compte du contexte spécifique de chaque application.

Mots-clés : Sécurité des logiciels, Dépendances logicielles, Vulnérabilités, Analyse statique du code source, Contextualisation, Java.

Abstract

The development of modern applications often requires the integration of various dependencies, whether open source or proprietary. These third-party components, such as libraries, frameworks, and modules, are used to accelerate development, add advanced features, and avoid reinventing the wheel.

However, these dependencies may contain vulnerabilities, whether known or unknown, that could be exploited by attackers. Such flaws can jeopardize the security of the application, compromise sensitive data, disrupt its normal operation, or even allow full control of the system on which it runs.

The aim of this work is to explore the mechanisms for contextualizing vulnerable dependencies based on source code analysis. This contextualization helps developers understand in depth the specific risks associated with each dependency. It also facilitates the prioritization of corrective actions for detected vulnerabilities, focusing on those that pose the greatest threat to the application.

This research seeks to examine current methods for contextualizing vulnerable software dependencies in a Java application, highlighting the shortcomings of existing methods and aiming to contribute to improving the contextualization of vulnerable dependencies through the use of source code analysis techniques.

The proposed solution was subjected to various types of tests to ensure its robustness and effectiveness. These tests confirmed the approach's ability to accurately contextualize vulnerable dependencies, while taking into account the specific context of each application.

Keywords : Software Security, Software Dependencies, Vulnerabilities, Static Code Analysis, Contextualization, Java.

ملخص

تتطلب عملية تطوير التطبيقات الحديثة غالباً دمج العديد من الاعتماديات، سواء كانت مفتوحة المصدر أو ملكية. تُستخدم هذه المكونات الخارجية، مثل المكتبات والأطر والنماذج، لتسريع عملية التطوير، إضافة ميزات متقدمة، وتجنب إعادة تطوير الحلول من الصفر.

ومع ذلك، قد تحتوي هذه الاعتماديات على ثغرات، سواء كانت معروفة أم غير معروفة، قد يستغلها المهاجمون. يمكن أن تعرض هذه الثغرات أمان التطبيق للخطر، تعرض البيانات الحساسة للخطر، تعطل عمله الطبيعي، أو حتى تسمح بالتحكم الكامل في النظام الذي يعمل عليه.

الهدف من هذا العمل هو استكشاف آليات سياق الاعتماديات الضعيفة استناداً إلى تحليل الشيفرة المصدرية. يساعد هذا السياق المطورين على فهم المخاطر المحددة المرتبطة بكل اعتماد بشكل عميق. كما يُسهِّل أيضاً تحديد أولويات الإجراءات التصحيحية للثغرات المكتشفة، مع التركيز على تلك التي تشكل أكبر تهديد للتطبيق.

تسعى هذه الدراسة إلى فحص الطرق الحالية لتسليط الضوء على نقاط الضعف في التبعيات البرمجية ضمن تطبيق جافا، مع إبراز أوجه القصور في الطرق الحالية وتهدف إلى المساهمة في تحسين تسليط الضوء على التبعيات الضعيفة من خلال استخدام تقنيات تحليل الشيفرة المصدرية.

تم إخضاع الحل المقترح لعدة أنواع من الاختبارات لضمان قوته وفعاليته. وقد أكدت هذه الاختبارات قدرة النهج على وضع سياق دقيق للتبعيات الضعيفة، مع الأخذ في الاعتبار السياق الخاص بكل تطبيق.

كلمات مفتاحية :أمان البرمجيات، الاعتماديات البرمجية، الثغرات، تحليل الشيفرة المصدرية، السياق، جافا

Table des matières

Dédicace . . . . . **I**

Remerciements . . . . . **II**

Résumé . . . . . **III**

Abstract . . . . . **IV**

ملخص . . . . . **V**

Introduction générale **1**

1 Concepts fondamentaux autour des dépendances logicielles Vulnérables 4
1.1 Introduction 5
1.2 Dépendances logicielles 5
1.3 Vulnérabilité des dépendances logicielles 6
1.4 Système de référence CVE 7
1.5 Analyse du code source 9
1.5.1 Analyse statique 9
1.5.2 Analyse dynamique 12
1.6 Outils d'analyse de composition logicielle 12
1.6.1 Éléments scannés par les outils SCA 13
1.6.2 Métriques utilisées par les outils SCA 14
1.7 Conclusion 17

2 Contextualisation des Dépendances Logicielles Vulnérables 19
2.1 Introduction 20
2.2 Littérature sur l'identification des vulnérabilités 20
2.2.1 Correspondance des vulnérabilités basée sur les noms 20
2.2.2 Correspondance des vulnérabilités basée sur le code 21
2.2.3 Correspondance des vulnérabilités basée sur le web sémantique . . 22
2.3 Littérature sur la prédiction de la criticité des vulnérabilités 24
2.4 Conclusion 28

3 Méthodologie de Développement et Analyse de Besoins 29
3.1 Introduction 30
3.2 Méthodologie de développement 30
3.2.1 Présentation de la méthodologie choisie 30
3.2.2 Motivation du choix de la méthode Scrum 32

3.3 Analyse de besoins . . . 33
3.3.1 Identification des acteurs du système . . . 33
3.3.2 Spécifications du système . . . 33
3.3.3 Diagramme de cas d'utilisation . . . 34
3.3.4 Diagramme d'activité . . . 37
3.4 Conclusion . . . 38

4 Conception . . . 39
4.1 Introduction . . . 40
4.2 Vue globale de la solution . . . 40
4.3 Conception détaillée . . . 42
4.3.1 Construction de la base de données des vulnérabilités . . . 42
4.3.2 Prédiction de la criticité des vulnérabilités . . . 46
4.3.3 Extraction de la liste des dépendances . . . 49
4.3.4 Analyse statique du code source . . . 51
4.3.5 Détection des vulnérabilités . . . 52
4.3.6 Priorisation des vulnérabilités détectées . . . 53
4.4 Conclusion . . . 55

5 Réalisation et Tests . . . 56
5.1 Introduction . . . 57
5.2 Technologies utilisées . . . 57
5.2.1 FastAPI . . . 57
5.2.2 MySQL . . . 57
5.2.3 Java . . . 58
5.2.4 Syft . . . 59
5.3 Présentations des outils utilisés . . . 59
5.3.1 Visual Studio Code . . . 59
5.3.2 SQL Server . . . 60
5.3.3 Postman . . . 60
5.3.4 Git . . . 60
5.3.5 Github . . . 61
5.3.6 kaggle . . . 61
5.4 Schéma technique de la solution . . . 61
5.5 Implémentation . . . 62
5.5.1 Intégration d'outils . . . 62
5.5.2 Implémentation des modèles de transfert learning . . . 63
5.6 Utilisation de l'application . . . 65
5.7 Tests . . . 67
5.7.1 Approche de test . . . 67
5.8 Limitations de la solution . . . 72
5.9 Conclusion . . . 72

Conclusion et perspectives . . . 73

Annexes . . . 77

A Contenu d'une vulnérabilité récupérée de la NVD 78

Table des figures

1.1 Exemple d'un arbre de dépendances [1]. 6
1.2 Application qui dépend de certaines dépendances vulnérables. 7
1.3 Contenu d'un CVE. [2] 8
1.4 Les informations sur la gravité de vulnérabilité [3] 9
1.5 Transformation du code source original en code sémantique [6] 10
1.6 Graphe de flux de contrôle du code sémantique [6] 10
1.7 Graphe de dépendance du code sémantique [6] 11
1.8 Exemple d'un graphe d'appels 12
1.9 Les éléments scannés par les outils SCA [10] 14
1.10 Les éléments scannés par les outils SCA [15] 15
1.11 Un exemple d'une application analysée en se basant sur la profondeur de la dépendance 16

2.1 Processus de correspondance des vulnérabilités basée sur les noms [16] . . 21
2.2 Détection des vulnérabilités en utilisant la correspondance des vulnérabilités basée sur le code [17] 22
2.3 Processus de correspondance des vulnérabilités basée sur le web sémantique [18] 23
2.4 prédiction de la criticité des vulnérabilités 26

3.1 Méthode Scrum [27] 31
3.2 Tableaux des tâches 32
3.3 Diagramme de cas d'utilisation du système 35
3.4 Diagramme d'activité du système 37

4.1 Paramètres d'Entrée et Résultats de Sortie de l'Outil à développer 40
4.2 Vue globale de la solution 41
4.3 Le point de terminaison pour récupérer les données de la NVD [4] 43
4.4 Schéma de notre base de données 43
4.5 Processus d'identification des commits correctifs des vulnérabilités 45
4.6 Le point de terminaison pour filtrer et renvoyer uniquement les CVEs modifiés [4] 45
4.7 Dataset 46
4.8 Architecture du modèle 49
4.9 Les formats normalisés de SBOM [29] 50
4.10 Processus d'extraction de l'AST 52

5.1 Logo FastAPI 57
5.2 Logo MySQL 57

5.3 Logo Java 58
5.4 Logo Syft 59
5.5 Logo Visual Studio Code 59
5.6 Logo SQL Server 60
5.7 Logo Postman 60
5.8 Logo Git 60
5.9 Logo Github 61
5.10 Logo Kaggle 61
5.11 Schéma technique de la solution 62
5.12 Division des Données 64
5.13 Entraînement de DistilBERT 65
5.14 Sélection de l'application Java en entrée 65
5.15 Lancement de la contextualisation 66
5.16 Visualisation du rapport 66
5.17 Exemple de résultat d'intégration de Syft 67
5.18 Résultat de Syft 68
5.19 Modifications de code liées au fix commit de CVE-2014-0050 69
5.20 Rapport de contextualisation des dépendances vulnérables de l'application étudiée 70

A.1 Contenu d'une vulnérabilité récupérée de la NVD 79

Liste des tableaux

1.1 Outils SCA 13
1.2 Les champs du système CVSS v3 [3] 17

2.1 Approches de prédiction de la criticité des vulnérabilités 27

3.1 Spécifications fonctionnelles du système 34
3.2 Spécifications techniques du système 34
3.3 Documentation de CU "Lancer le processus de contextualisation" 36
3.4 Documentation de CU "Visualiser les résultats de contextualisation" 36

4.1 La distribution des vulnérabilités selon les catégories du CVSS 47
4.2 Les différentes informations fournies par chacun de deux formats SPDX et CycloneDX [29]. 51
4.3 Les valeurs des métriques CVSS 54

5.1 Test et évaluation des modèles de prédiction des métriques CVSS 71

Liste des algorithmes

1 DétecterVulnérabilités(CodeSource, BaseDeDonnéesVulnérabilités) 53

Liste des sigles et acronymes

API	*Application Programming Interface*
CVE	*Common Vulnerabilities and Exposures*
CVSS	*Common Vulnerability Scoring System*
CFG	*Control Flow Graph*
CG	*Call Graph*
SCA	*Software Composition Analysis*
CPE	*Common Platform Enumeration*
AST	*Abstract Syntax Tree*
POM	*Project Object Model*
BERT	*Bidirectional Encoder Representations from Transformers*
SBOM	*Software Bill of Materials*
SPDX	*Software Product Data Exchange*
CNN	*Convolutional Neural Network*

Introduction générale

Contexte

Le développement d'une application nécessite généralement l'intégration de diverses dépendances, qu'elles soient open source ou propriétaires. Ces dépendances sont des composants tiers, tels que des bibliothèques, des frameworks ou des modules qui sont intégrés dans le code source d'une application pour accélérer le processus de création, intégrer des fonctionnalités avancées et éviter de développer les logiciels à partir de zéro.

Ces dépendances peuvent contenir des vulnérabilités connues ou inconnues qui pourraient être exploitées par des attaquants pour compromettre la sécurité de l'application et accéder à des données sensibles, perturber le fonctionnement normal de l'application ou même prendre le contrôle du système sur lequel elle s'exécute.

La contextualisation des dépendances vulnérables consiste à fournir aux développeurs une compréhension approfondie des risques spécifiques que chaque dépendance vulnérable représente pour leur application afin de les aider à prioriser les actions de correction des vulnérabilités détectées.

L'analyse statique du code source représente une stratégie fondamentale dans la recherche des dépendances vulnérables exploitables au sein des applications. Cette méthode vise à détecter les dépendances vulnérables qui pourraient être exploitées par des attaquants en visualisant le code source de l'application analysée.

L'analyse dynamique du code source constitue une approche complémentaire à l'analyse statique, offrant une perspective en temps réel sur le comportement d'une application lors de son exécution. Cette méthode permet de détecter les dépendances vulnérables qui pourraient être exploitées lors de l'exécution de l'application.

Problématique

Le développement d'une application implique généralement d'utiliser des dépendances (open sources ou non) qui peuvent être associées à des vulnérabilités pour une version donnée, laissant un besoin crucial d'explorer les approches d'analyse des composants logiciels vulnérables dans une application développée afin d'assister les développeurs dans la priorisation des actions de correction des vulnérabilités.

Dans ce travail, nous allons proposer une solution sous la forme d'un outil qui contextualise les dépendances vulnérables dans les applications Java. Cette contextualisation consiste à détecter les dépendances logicielles vulnérables dans ces applications et à prioriser les vulnérabilités en fonction de leur risque pour l'application à analyser.

Objectifs

Pour répondre à cette problématique, nous allons concevoir et implémenter un outil, Pradeo Analyzer, permettant de contextualiser les dépendances logicielles vulnérables dans une application Java à analyser. Les objectifs à atteindre sont donc :

1. Concevoir une base de données complète sur les vulnérabilités des dépendances logicielles.
2. Mettre en place des techniques de détection des dépendances logicielles vulnérables en se basant sur l'analyse statique du code source.
3. Mettre en place des techniques de priorisation des vulnérabilités afin d'aider les développeurs à prioriser les actions de correction de ces vulnérabilités.

Organisation du mémoire

Ce mémoire est scindé en 5 chapitres :

1. Le premier chapitre, intitulé « **Concepts Fondamentaux autour des Dépendances Logicielles Vulnérables** », a pour objectif de mettre en avant certains concepts et définitions liés aux dépendances logicielles vulnérables.
2. Le deuxième chapitre, intitulé « **Contextualisation des Dépendances Logicielles Vulnérables** », vise à examiner les méthodes de contextualisation, y compris la détection et la priorisation des vulnérabilités des dépendances.
3. Le troisième chapitre, intitulé « **Méthodologie de Développement et Analyse des Besoins** », présente les besoins de notre projet ainsi que la méthodologie de développement suivie pour atteindre les objectifs du projet.
4. Le quatrième chapitre, intitulé « **Conception** », modélise la solution proposée en détaillant les différents composants de l'outil ainsi que leurs interactions pour répondre aux besoins identifiés..
5. Le cinquième chapitre, intitulé « **Réalisation et Tests** », décrit le processus de mise en œuvre de la solution proposée. Il couvre les différentes étapes de développement ainsi que les tests réalisés pour évaluer l'efficacité et la fiabilité de l'outil.

Le mémoire se termine par une conclusion générale visant à synthétiser les points clés abordés tout au long du travail, tout en offrant des perspectives pour les développements futurs.

Chapitre 1

Concepts fondamentaux autour des dépendances logicielles Vulnérables

1.1 Introduction

Les dépendances logicielles jouent un rôle crucial dans le développement moderne d'applications. Elles permettent aux développeurs d'intégrer rapidement des fonctionnalités avancées en utilisant des bibliothèques, des frameworks et des modules créés par des entités tierces. Cependant, elles peuvent aussi introduire des vulnérabilités si elles ne sont pas correctement gérées. Les cyber attaquants exploitent fréquemment ces failles pour compromettre la sécurité des applications, accéder à des données sensibles ou prendre le contrôle des systèmes.

Pour minimiser ces risques, il est essentiel de gérer activement les dépendances logicielles. Cela peut être facilité par divers outils disponibles qui contextualisent les dépendances vulnérables en détectant et en évaluant ces vulnérabilités, en utilisant différentes méthodologies telles que l'analyse du code source de l'application concernée (analyse statique) et l'analyse de son comportement lors de l'exécution (analyse dynamique).

Ce chapitre propose une vue d'ensemble des dépendances logicielles, en se concentrant sur la vulnérabilité de ces dernières et sur les méthodes d'analyse du code source, qu'elles soient statiques ou dynamiques. Il explore également les outils d'analyse de composition logicielle et les métriques utilisés pour évaluer les risques associés aux dépendances vulnérables.

1.2 Dépendances logicielles

Les dépendances logicielles sont des composants utilisés par un logiciel pour fournir certaines fonctionnalités ou pour interagir avec des systèmes externes. Ces dépendances peuvent être des bibliothèques, des modules ou des API intégrés dans le projet logiciel [1].

Ces dépendances peuvent être directes comme elles peuvent être transitives. Les dépendances directes sont directement appelées depuis l'instance de la bibliothèque dépendante. Par contre, les dépendances transitives sont des dépendances indirectes qui ne sont pas directement référencées dans le code source de la bibliothèque dépendante, mais qui sont nécessaires par le biais d'autres dépendances directes comme illustrées dans la Figure 1.1 [1].

Les entreprises peuvent développer leurs propres dépendances pour les utiliser dans leurs projets, tout comme elles peuvent utiliser des dépendances externes à l'entreprise si nécessaire.

La Figure 1.1 [1] illustre un arbre de dépendances pour une application M. Cette application dépend de trois dépendances directes : m2, x1 et y1. Plus précisément, x1 dépend de u1, qui est une dépendance transitive par rapport à l'application M. De même, y1 dépend de y2, qui est une dépendance transitive par rapport à l'application M et qui dépend à son tour de la dépendance transitive z1.

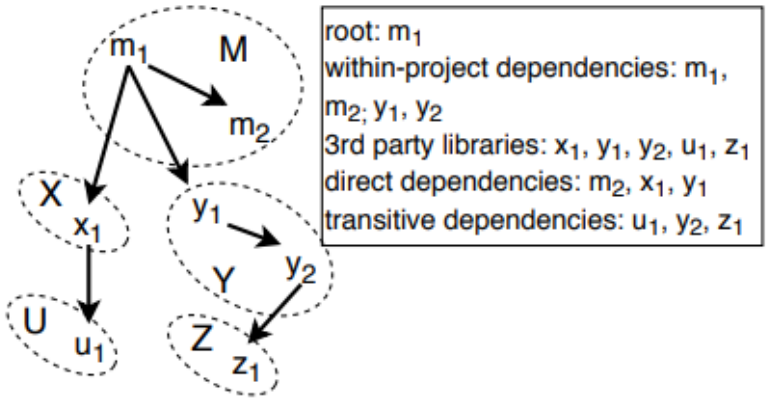


FIG. 1.1 : Exemple d'un arbre de dépendances [1].

1.3 Vulnérabilité des dépendances logicielles

Les dépendances logicielles peuvent être vulnérables, c'est-à-dire peuvent contenir des failles de sécurité connues ou potentielles. On parle dans ce cas de dépendances vulnérables. Ces dépendances peuvent poser un risque sérieux pour la sécurité d'une application ou d'un système, car elles offrent une surface d'attaque potentielle aux cybercriminels [1].

Les dépendances vulnérables surviennent principalement pour trois raisons :

- **Utilisation de composants obsolètes ou non mis à jour :** Lorsqu'un composant logiciel n'est pas régulièrement mis à jour, il devient susceptible de contenir des failles de sécurité connues qui n'ont pas été corrigées. Cela peut arriver parce que les développeurs ne surveillent pas les mises à jour des composants ou les ignorent par manque de temps ou de ressources.[1]

- **Manque de vérification des dépendances tierces pour les failles de sécurité :** Les dépendances tierces, souvent utilisées pour accélérer le développement en réutilisant du code existant, peuvent introduire des vulnérabilités si elles ne sont pas régulièrement vérifiées. Les développeurs peuvent ne pas disposer des outils ou des processus nécessaires pour identifier les failles potentielles dans ces bibliothèques externes.[1]

- **Absence de gestion appropriée des bibliothèques externes :** Une gestion inadéquate des bibliothèques externes, comme le manque de suivi des versions et de leur compatibilité, peut entraîner l'intégration de versions vulnérables dans le projet. Sans une politique claire de gestion des dépendances, il devient difficile de maintenir un environnement sécurisé.[1]

La Figure 1.2 illustre une application qui dépend directement de deux bibliothèques : Apache POI et Google Gson, toutes deux vulnérables. Apache POI, en plus de sa propre

vulnérabilité, a des dépendances transitives sur Commons Codec, qui est également vulnérable, et Apache Commons, qui est non vulnérable. Cela met en évidence que, malgré la présence de dépendances sécurisées comme Apache Commons, la vulnérabilité des autres composants, y compris les dépendances directes et transitives, peut compromettre la sécurité globale de l'application.

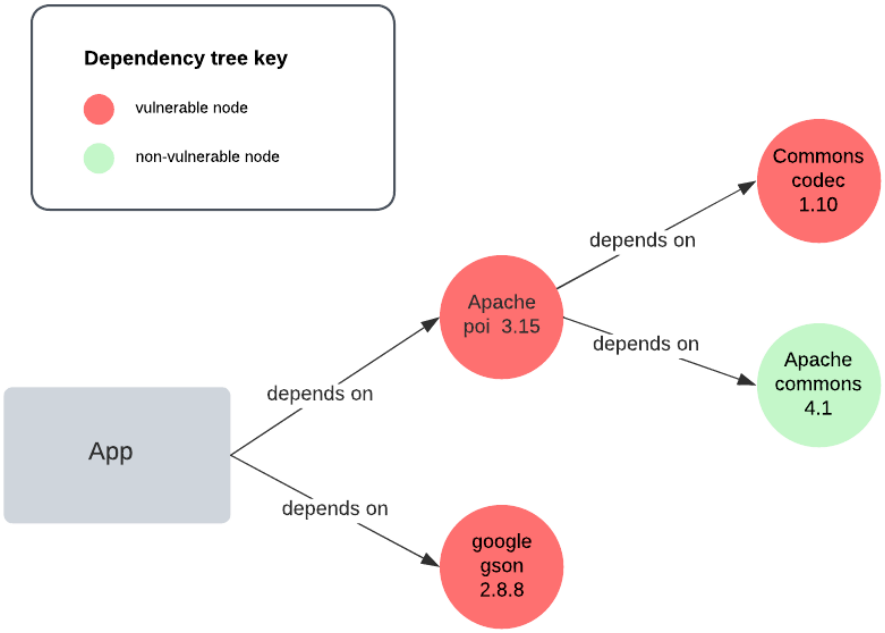


Fig. 1.2 : Application qui dépend de certaines dépendances vulnérables.

Pour atténuer les risques associés aux dépendances vulnérables, il est essentiel de mettre en place des pratiques de développement sécurisé, telle que l'utilisation d'outils d'analyse statique et dynamique pour détecter les failles de sécurité potentielles. Mais avant de présenter ces techniques d'analyse, nous allons d'abord définir de manière précise le concept de CVE (Common Vulnerabilities and Exposures).

1.4 Système de référence CVE

Les vulnérabilités logicielles sont répertoriées et suivies à travers le système CVE (Vulnérabilités et Expositions Communes) [2], un cadre de gestion utilisé pour identifier et cataloguer les vulnérabilités de sécurité dans les logiciels et systèmes informatiques. Chaque vulnérabilité, appelée CVE, reçoit un identifiant unique, ce qui permet de la référencer et de la suivre de manière standardisée. Les CVE [2] fournissent une manière standardisée de référencer et de suivre les vulnérabilités, ce qui facilite la communication entre les différentes parties prenantes, y compris les chercheurs en sécurité, les entreprises, les développeurs de logiciels et les utilisateurs finaux.

Comme illustré dans la Figure 1.3, chaque CVE (vulnérabilité) [2] est accompagné d'un identifiant unique attribué à la vulnérabilité et d'une description détaillée de la vul-

nérabilité. L'identifiant du CVE est formaté sous la forme "CVE-yyyy-nnnnn", où "yyyy" représente l'année de publication de la CVE et "nnnnn" est un numéro séquentiel attribué à la vulnérabilité. Cet identifiant permet d'identifier de manière unique une vulnérabilité, facilitant ainsi sa référence et son suivi par les chercheurs en sécurité et les professionnels de la cybersécurité. [3]

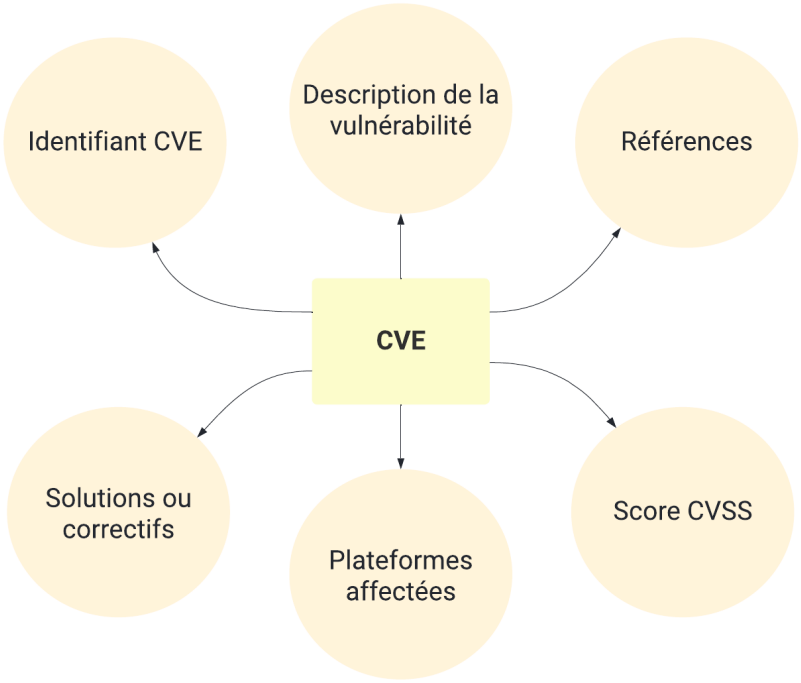


Fig. 1.3 : Contenu d'un CVE. [2]

Les CVE [2] peuvent également inclure des informations sur les références aux sources d'information supplémentaires, ainsi que sur les solutions ou correctifs disponibles.

Les plateformes affectées sont représentées par Le champ CPE (Common Platform Enumeration) [3] qui est utilisé pour identifier de manière normalisée des produits, plateformes ou configurations logicielles qui sont affectés par la vulnérabilité concernée. Le CPE utilise un format hiérarchique pour décrire ces éléments de manière structurée. Il comprend généralement plusieurs parties, notamment :

GroupID : Identifie le groupe auquel appartient le produit ou la plateforme. Il est généralement utilisé pour regrouper les projets liés [4].

ArtifactID : Identifie de manière unique l'artefact (ou module) au sein du groupe. C'est le nom du produit [4].

Version : Identifie la version spécifique du produit [4].

Les CVE peuvent inclure aussi des informations sur la gravité de la vulnérabilité (Score CVSS [3]) qui est généralement attribuée en fonction de critères spécifiques tels que le potentiel d'impact et la facilité d'exploitation. La Figure 1.4 [3] présente les champs du système CVSS , lesquels comprennent l'impact potentiel d'une vulnérabilité sur la confidentialité, l'intégrité et la disponibilité des données. Ces champs comprennent aussi le vecteur d'attaque , qui indique comment l'attaque est réalisée,et la complexité d'attaque,

qui évalue la difficulté d'exploitation de la vulnérabilité , le Scope qui indique l'étendue de l'impact de la vulnérabilité, et enfin, les Privilèges Requis, qui indiquent les privilèges nécessaires pour être capable de lancer l'attaque [3].

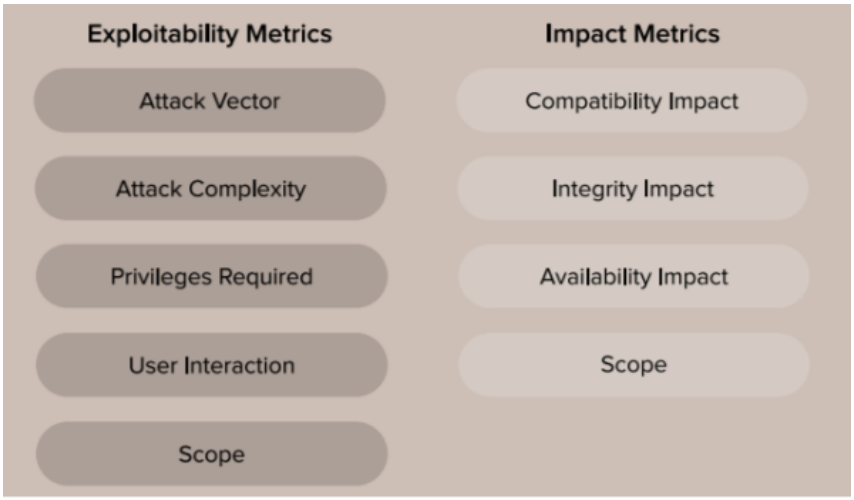


Fig. 1.4 : Les informations sur la gravité de vulnérabilité [3]

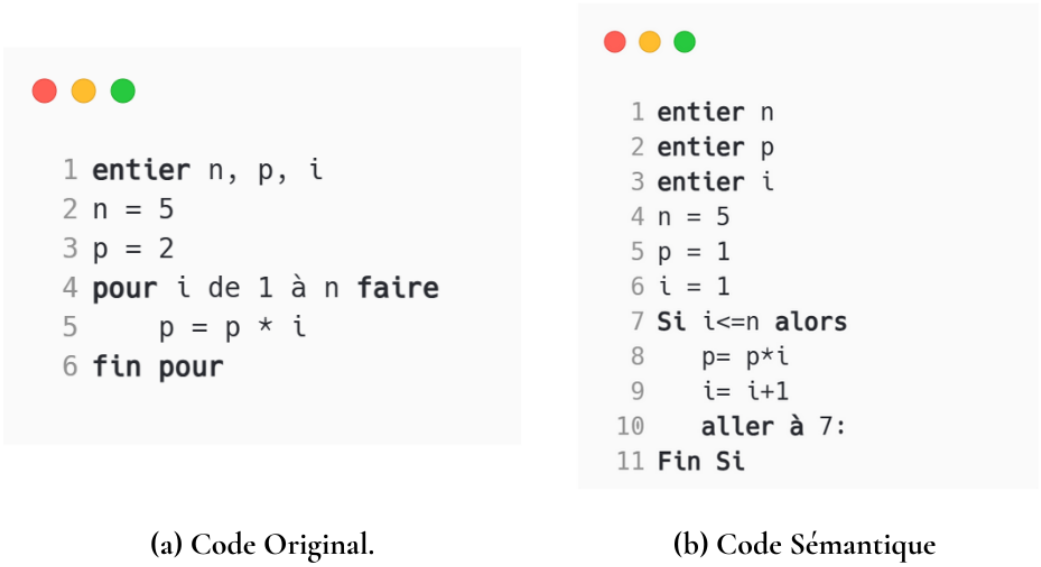
1.5 Analyse du code source

L'analyse du code revêt une grande importance pour identifier les dépendances potentiellement vulnérables et contextualiser ces dépendances en déterminant quelles CVE (Common Vulnerabilities and Exposures) [2] sont exploitables en fonction de leur utilisation dans l'application développée. Cette approche aide les développeurs à se concentrer sur les aspects critiques de la sécurité des applications. L'utilisation combinée d'analyses statiques et dynamiques [5] permet d'obtenir une vue d'ensemble des vulnérabilités et des risques associés au code, facilitant ainsi la détection et la correction préventive des vulnérabilités.

1.5.1 Analyse statique

Plusieurs techniques d'analyse pour l'analyse statique existent : l'analyse de flux de contrôle, l'analyse du graphe de dépendances et l'analyse du graphe d'appels .

Avant d'établir le graphe de flux de contrôle, une transformation du code source original en code sémantique est nécessaire afin de simplifier la complexité du code source tout en préservant sa sémantique essentielle comme montré dans la Figure 1.5 [6].



```
1 entier n, p, i
2 n = 5
3 p = 2
4 pour i de 1 à n faire
5     p = p * i
6 fin pour
```

(a) Code Original.

```
1  entier n
2  entier p
3  entier i
4  n = 5
5  p = 1
6  i = 1
7  Si i<=n alors
8     p= p*i
9     i= i+1
10    aller à 7:
11 Fin Si
```

(b) Code Sémantique

FIG. 1.5 : Transformation du code source original en code sémantique [6]

L'analyse de flux de contrôle en se basant sur le graphe de flux de contrôle (CFG) [6] est utilisé principalement dans le domaine de l'analyse statique des programmes pour comprendre la manière dont les instructions d'un programme sont exécutées les unes après les autres ou chaque nœud est associé à une instruction, une opération ou un bloc de code et chaque arête représente le flux de contrôle entre les différentes instructions (Elles indiquent l'ordre d'exécution des instructions). Les structures de contrôle conditionnel telles que les "if" et les "switch" sont représentées dans le CFG par des arêtes divergentes qui indiquent les différentes branches conditionnelles possibles.

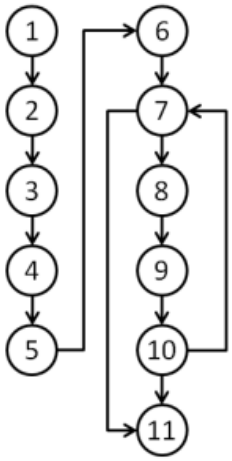


FIG. 1.6 : Graphe de flux de contrôle du code sémantique [6]

Comme montré dans la Figure 1.6 [6] . Cet exemple illustre le graphe de flux de contrôle du code sémantique présenté dans la Figure 1.5. Pour clarifier, la présence d'une

arête entre les nœuds 1 et 2 indique que la deuxième instruction s'exécute immédiatement après la première. De même, il existe une arête entre les nœuds 10 et 7 car la dixième instruction contient un saut inconditionnel vers l'instruction numéro 7.

Le graphe de dépendances (DG) [6] est l'une des méthodes existantes pour réaliser l'analyse statique du code. Dans ce type de graphe, une arête de dépendance est établie entre deux instructions dans le graphe si une variable particulière est utilisée (read) dans la première instruction, tandis que la même variable est assignée (write) dans la seconde instruction. La Figure 1.7 [6] présente le graphe de dépendances du code sémantique illustré dans la Figure 1.5, où les nœuds représentent les instructions du code. Chaque arête dans le graphe indique qu'une variable est utilisée en mode lecture dans l'instruction source et qu'elle est utilisée en mode écriture dans l'instruction destination. Par exemple, une arête entre le nœud 1 et le nœud 4 pourrait représenter le fait que la variable n est déclarée dans l'instruction 1 et assignée dans l'instruction 4.

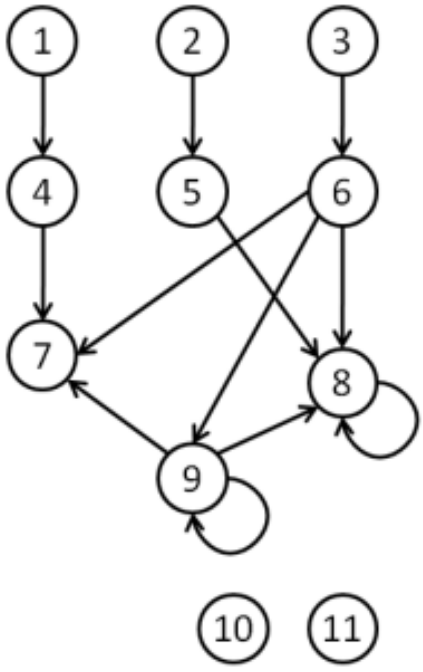


Fig. 1.7 : Graphe de dépendance du code sémantique [6]

Le graphe d'appels (CG) [6] représente les relations d'appel entre les fonctions ou méthodes d'un programme. Chaque nœud du graphique représente une fonction, et les arêtes entre les nœuds indiquent les appels de fonction. Comme illustré dans la Figure 1.8, il y a deux arêtes sortantes de la fonction « main » vers les fonctions « calculate » et « println », car la fonction « main » fait appel à ces deux fonctions. De même, il y a également deux arêtes sortantes de la fonction « calculate » vers les fonctions « getSecond » et « getFirst », car la fonction « calculate » fait appel à ces deux fonctions, une arête existe également entre « getSecond » et « println » car « getSecond » fait appel à « println ».

```
void main() {
int x = calculate();
System.out.println(x);
}
int calculate() {
int a = getFirst();
int b = getSecond();
return a + b; }
int getFirst() { return 12; }
int getSecond() {
System.out.println("second");
return 30; }
```

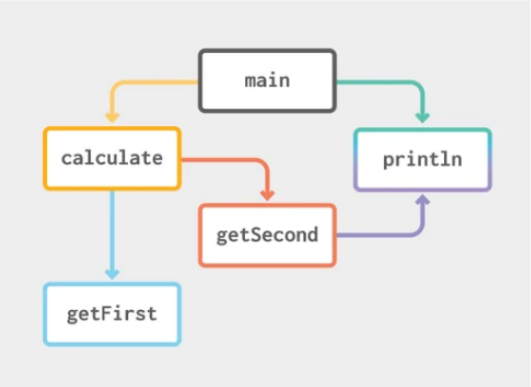


FIG. 1.8 : Exemple d'un graphe d'appels

Limitations de l'analyse statique : L'analyse statique citée par [5] présente certaines limitations lorsqu'il s'agit d'évaluer les risques présentés par les dépendances vulnérables. D'abord, les analyses de code statique se basent uniquement sur le code source sans exécuter réellement le programme. Cela signifie qu'elles ne tiennent pas compte des informations qui pourraient être déduites uniquement pendant l'exécution du programme. De plus, l'analyse statique peut parfois produire des résultats imprécis en raison de la complexité du code et des nombreuses possibilités d'exécution.

1.5.2 Analyse dynamique

L'analyse dynamique implique l'examen du code avec des traces et des chemins d'exécution.

La collecte des exécutions réelles des éléments peut être effectuée à différents moments : lors de tests unitaires, de tests d'intégration, et même pendant le fonctionnement en direct du système (si possible) [7].

L'analyse dynamique vise a évaluer si les éléments vulnérables sont accessibles en observant les exécutions réelles.

Limitations de l'analyse dynamique : L'analyse dynamique dépend totalement de l'exploration de chemins d'exécution spécifiques pour fournir des informations précises sur le comportement réel d'un logiciel. Cela rend la couverture de code plus faible.

1.6 Outils d'analyse de composition logicielle

SCA (Software Composition Analysis) [8] est un processus qui vise à examiner et à évaluer les composants logiciels utilisés dans une application ou un produit logiciel en extrayant les composants logiciels (les dépendances) qui présentent des vulnérabilités.

Il existe de nombreuses outils SCA disponibles pour aider les développeurs à identifier et à gérer les vulnérabilités dans les dépendances logicielles. La table 1.1 ci-dessous

présente quelques-uns des outils les plus cités.

Outil	Projets analysés	Techniques d'analyse	Description
OWASP Dependency-Check [9]	Projets Maven, npm, gradle	Analyse de manifestes	Il détecte les vulnérabilités présentes dans les dépendances d'un projet en analysant le fichier manifeste (le fichier de construction de l'ap- plication) et en se basant sur la base de données NVD (National Vulnerability Database) [4]
Trivy [10]	Projets Java,Python,npm ...	Analyse de manifestes	Il se base sur une analyse des fichiers de manifeste (le fichier de construction de l'application) pour identifier les dépendances et les vulnérabilités potentielles. Il compare les packages et les bibliothèques présentes dans l'image du conteneur [11] ou le système de fichiers avec sa base de données de vulnérabilités.
Eclipse Steady [12]	Projets Java	Analyse Statique et Analyse Dynamique	Il collecte des preuves sur l'exécution de code vulnérable dans un contexte d'application donné (grâce à la combinaison de techniques d'analyse statique et dynamique)
Grype [13]	Projets Java,Python,npm ...	Analyse de manifestes	Il constitue un scanner de vulnérabilités pour les images de conteneurs [11] et les systèmes de fichiers, détectant les dépendances vulnérables référencées dans les fichiers de manifeste.
Black Duck [14]	Projets Java, .NET et Python	Analyse Statique	Il effectue une analyse du code pour comprendre comment les différentes vulnérabilités peuvent se propager dans l'application en fonction des relations de dépendance entre les différents composants

Tab. 1.1 : Outils SCA

1.6.1 Éléments scannés par les outils SCA

Les outils SCA, appelés aussi les scanners de vulnérabilités open source, sont conçus pour détecter les vulnérabilités dans différents types d'artefacts, dont les images de conteneurs, les systèmes de fichiers, les dépôts Git, et bien d'autres comme montré dans la Figure 1.9 [10].

Images de Conteneurs : les outils SCA peuvent scanner les images de conteneurs en analysant les couches et les dépendances au sein de l'image. Il vérifie les vulnérabilités présentes dans les paquets et les bibliothèques installés en les comparant à une base de données de vulnérabilités régulièrement mise à jour. Ils prennent en charge différents

formats d'images de conteneurs, tels que Docker [11].

Systèmes de Fichiers : Pour un système de fichiers classique, les outils peuvent être utilisés pour scanner les fichiers et les bibliothèques à l'intérieur d'un répertoire spécifié. Ils fonctionnent en inspectant le contenu des fichiers et des répertoires, en vérifiant la présence de vulnérabilités connues dans les composants logiciels inclus. Cela peut être utile pour analyser les machines hôtes ou des répertoires individuels à la recherche de problèmes de sécurité [10].

Dépôts Git (à distance) : les outils SCA [8] peuvent également scanner les dépôts Git en analysant le contenu de la base de code. Ils vérifient les dépendances spécifiées dans les fichiers du projet (par exemple, requirements.txt pour Python, package.json pour Node.js) et les scanne à la recherche de vulnérabilités. Cela aide à identifier les problèmes de sécurité dans le code source et ses dépendances [10].

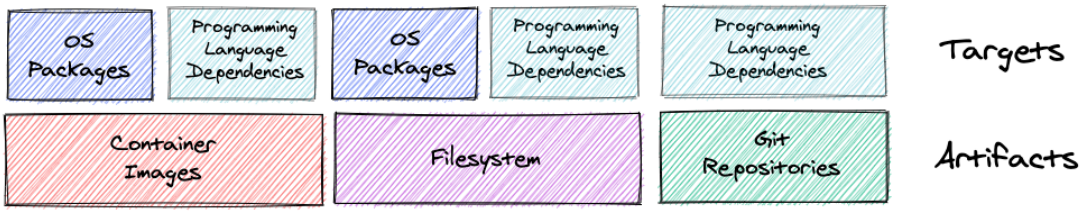


Fig. 1.9 : Les éléments scannés par les outils SCA [10]

1.6.2 Métriques utilisées par les outils SCA

Lorsqu'une vulnérabilité est présente dans une dépendance, il est essentiel de contextualiser le risque de cette vulnérabilité en fonction de la manière dont l'application utilise cette dépendance. Cette contextualisation des dépendances vulnérables permet de déterminer si une CVE spécifique est effectivement exploitable dans le contexte particulier de l'application. Nous avons observé que les outils SCA étudiés fournissent plusieurs métriques dans leurs rapports d'analyse pour soutenir ce type d'évaluation contextuelle. Nous classifions ces métriques en trois catégories selon [8].

Métriques basées sur l'analyse du code

Les outils peuvent analyser le code source ou les binaires pour déduire l'utilisation de la dépendance et la portée de la vulnérabilité [8].

Analyse de la portée (Reachability Analysis) : L'outil peut enrichir sa base de données de vulnérabilités avec des détails sur la partie du code (par exemple, méthode, classe) impliquée dans une vulnérabilité spécifique [5]. Les outils peuvent ensuite déduire si le code vulnérable est accessible depuis l'application dépendante par le biais d'une analyse statique et/ou dynamique. La figure 1.10 montre un exemple de 4 projets qui dépendent de la dépendance vulnérable commons-io. Pour pmd-core et openhub-core, ils peuvent accéder à la fonction vulnérable getPrefixLength() de manière transitive. En revanche, les deux autres projets (c'est-à-dire, github-api et runtime-domain-sqlg) ne peuvent pas accéder à la fonction vulnérable par tous les moyens. Les outils existants comme OWASP

Dependency-Check signaleront que tous les quatre projets sont vulnérables car ils ignorent la possibilité d'accéder au code vulnérable, ce qui entraîne ainsi des faux positifs [15].

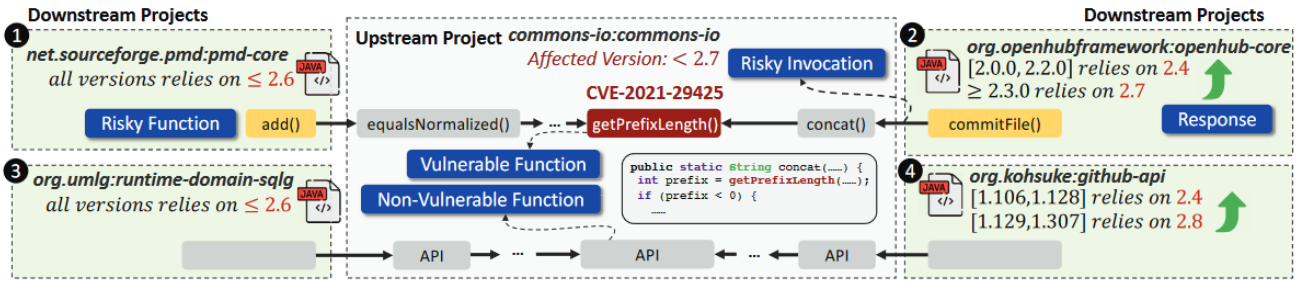


FIG. 1.10 : Les éléments scannés par les outils SCA [15]

Métriques basées sur les caractéristiques de la dépendance

La portée et la profondeur de la dépendance peuvent indiquer le risque de la vulnérabilité qu'elle contient dans le contexte de l'application.

- **Portée de la dépendance (Dependency Scope) :** est la manière dont une dépendance spécifique est utilisée dans un projet logiciel. Cela concerne généralement la partie du code où la dépendance est réellement invoquée ou utilisée. La portée indique dans quelles parties du projet la dépendance est active et a une influence. Les outils SCA excluent les dépendances utilisées exclusivement pendant le développement du processus de détection des vulnérabilités. Les dépendances réservées au développement sont celles qui ne sont utilisées que pendant le processus de développement et de test d'une application, tout comme les dépendances de portée test qui ne génèrent pas de vulnérabilités car elles ne sont pas déployées avec l'application finale. [8]

- **Profondeur de la dépendance :** Le risque peut être associé à la profondeur à laquelle une dépendance se situe dans l'arborescence des dépendances. [8]

  Les outils SCA qui tiennent compte de la profondeur des dépendances lors de l'analyse de l'application, comme illustré dans la Figure 1.11, estiment que le risque associé à la dépendance vulnérable A est plus élevé que celui de la dépendance vulnérable D. De plus, ils jugent que la probabilité de propagation de la vulnérabilité A est plus grande, car la dépendance A est directe et plus proche de l'application analysée dans l'arborescence des dépendances.

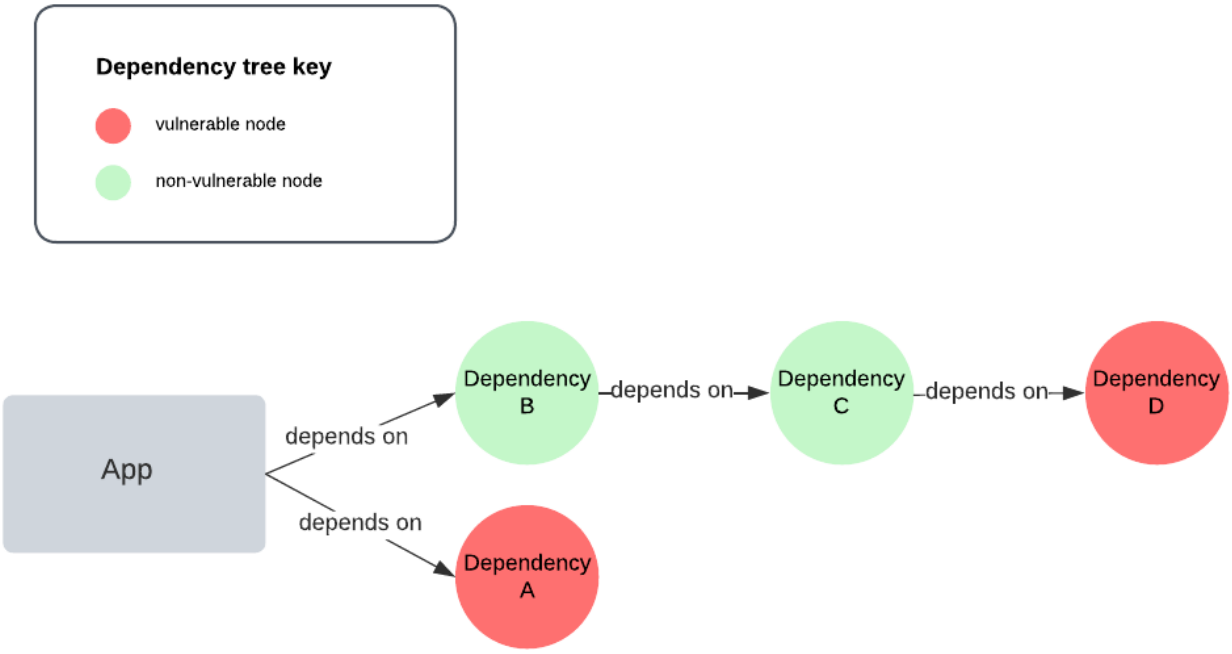


FIG. 1.11 : Un exemple d'une application analysée en se basant sur la profondeur de la dépendance

Métrique basée sur la vulnérabilité

Les caractéristiques de la vulnérabilité elle-même peuvent être utilisées pour évaluer le risque.

- **Popularité :** La popularité ou la notoriété d'une vulnérabilité peut indiquer la probabilité qu'elle soit exploitée. Une vulnérabilité largement recherchée et discutée est souvent plus susceptible d'être exploitée, car elle attire l'attention des attaquants potentiels et des chercheurs en sécurité. Pour évaluer cette popularité, l'outil effectue une analyse approfondie des tendances Google associées à chaque vulnérabilité. Cette analyse permet de mesurer le nombre de recherches effectuées au cours de la dernière période pour l'identifiant CVE spécifique [2]. En surveillant ces tendances, l'outil peut fournir des indications précieuses sur l'intérêt croissant ou décroissant pour une vulnérabilité donnée, ce qui peut aider à prioriser les efforts de remédiation en fonction de la visibilité et de la probabilité d'exploitation réelle dans le monde réel.

- **Gravité :** La norme de l'industrie pour évaluer la gravité des vulnérabilités est le Common Vulnerability Scoring System (CVSS) [3], qui est publiquement disponible pour les CVE [2], Le tableau 5.1 ci-dessous représente les champs CVSS v3 et leurs contenus [8].

Champ	Contenu
Attack Vector	Indique comment l'attaque est réalisée. Les valeurs possibles sont : Network (Réseau), Adjacent (Adjacent), Local (Local), Physical (Physique).
Attack Complexity	Indique la complexité pour faire l'attaque. Les valeurs possibles sont : Low (Faible), High (Élevée).
Privileges Required	Indique les privilèges requis pour être capable de lancer l'attaque. Les valeurs possibles sont : None (Aucun), Low (Faible), High (Élevée).
Confidentiality Impact	Indique l'impact sur la confidentialité des données si la vulnérabilité a été exploitée. Les valeurs possibles sont : None (Aucun), Low (Faible), High (Élevée).
Integrity Impact	Indique l'impact sur l'intégrité des données. Les valeurs possibles sont : None (Aucun), Low (Faible), High (Élevée).
Availability Impact	Indique l'impact sur la disponibilité du système. Les valeurs possibles sont : None (Aucun), Low (Faible), High (Élevée).
Scope	Indique l'étendue de l'impact de la vulnérabilité. Les valeurs possibles sont : Unchanged (Indique que l'exploitation de la vulnérabilité n'affecte que les ressources directement accessibles ou manipulées par l'application), Changed (Indique que l'exploitation de la vulnérabilité peut entraîner une modification de l'environnement ou des ressources qui sont en dehors de la portée directe de l'application).
Base Score	Le Score de base est un score attribué à une vulnérabilité en fonction des valeurs des autres champs du CVSS v3, il représente la gravité de la vulnérabilité.

Tab. 1.2 : Les champs du système CVSS v3 [3]

1.7 Conclusion

En somme, bien que les dépendances logicielles soient indispensables pour le développement rapide et efficace des applications, elles présentent également des risques de sécurité significatifs. L'utilisation d'outils spécialisés dans la contextualisation des dépendances vulnérables, capables de détecter et d'évaluer les vulnérabilités à l'aide de méthodologies telles que l'analyse statique et dynamique, permet d'identifier quelles vulnérabilités sont réellement accessibles et exploitables. En adoptant ces pratiques, les développeurs peuvent renforcer la sécurité de leurs applications, profiter des avantages des dépendances logicielles tout en minimisant les risques associés, et ainsi assurer la robustesse et la fiabilité des systèmes logiciels. Dans le prochain chapitre, nous allons discuter des approches de détection des dépendances vulnérables, ainsi que des méthodes de prédiction de la

criticité des vulnérabilités, permettant ainsi de les prioriser.

Chapitre 2

Contextualisation des Dépendances Logicielles Vulnérables

2.1 Introduction

La contextualisation des dépendances vulnérables consiste à identifier les dépendances vulnérables dans l'application à analyser et à les prioriser en fonction des risques qu'elles posent. Dans le contexte de l'identification des vulnérabilités des dépendances logicielles, de nombreuses approches pertinentes ont été proposées. Chacune de ces approches présente des avantages et des limites.

Dans ce chapitre, nous comparons ces travaux en fonction de plusieurs critères, notamment leurs objectifs, leurs entrées, leur processus d'identification et leurs résultats. Nous présentons également les approches de prédiction automatique de la criticité des vulnérabilités logicielles, qui visent à prioriser les vulnérabilités identifiées. Ces approches permettent d'optimiser les efforts de sécurité en se concentrant sur les vulnérabilités les plus critiques, assurant ainsi une meilleure protection des systèmes informatiques.

2.2 Littérature sur l'identification des vulnérabilités

De nombreuses approches pertinentes ont été proposées pour l'identification des vulnérabilités dans les dépendances logicielles. Chacune de ces approches [16] [17] [18] utilise sa propre méthode d'identification. Nous allons découvrir ces différentes approches, examiner leurs méthodologies d'identification, et analyser leurs avantages ainsi que leurs limites. Tout cela afin de mieux comprendre comment chaque méthode contribue à la sécurité des logiciels en identifiant efficacement les vulnérabilités.

2.2.1 Correspondance des vulnérabilités basée sur les noms

Un processus par lequel les vulnérabilités d'un logiciel ou d'un système sont identifiées en comparant les noms ou les étiquettes des vulnérabilités connues avec ceux des composants ou des bibliothèques utilisés dans le logiciel ou le système en question [16].

Par exemple, si une vulnérabilité connue est identifiée sous le nom de "Vulnérabilité X", un processus de correspondance des vulnérabilités basé sur les noms vérifierait si un composant ou une bibliothèque utilisée dans le logiciel contient également une référence à "Vulnérabilité X" dans la base de données des vulnérabilités. Si une telle correspondance est trouvée, cela indiquerait qu'une "Vulnérabilité X" connue est présente dans le logiciel.

La figure 2.1 montre comment appliquer la correspondance des vulnérabilités basée sur les noms pour détecter les vulnérabilités dans les dépendances d'un projet donné. Ce processus permet de signaler des alertes sur la présence de vulnérabilités potentielles sans avoir besoin d'examiner tout le code source du logiciel. Il repose sur deux éléments en entrée : le projet logiciel à analyser et les données de vulnérabilité (base de données des CVE). Ensuite, le vérificateur de vulnérabilités utilise ces entrées pour extraire la liste des dépendances du projet, les identifier et les faire correspondre avec les vulnérabilités connues en utilisant la base de données des CVE.

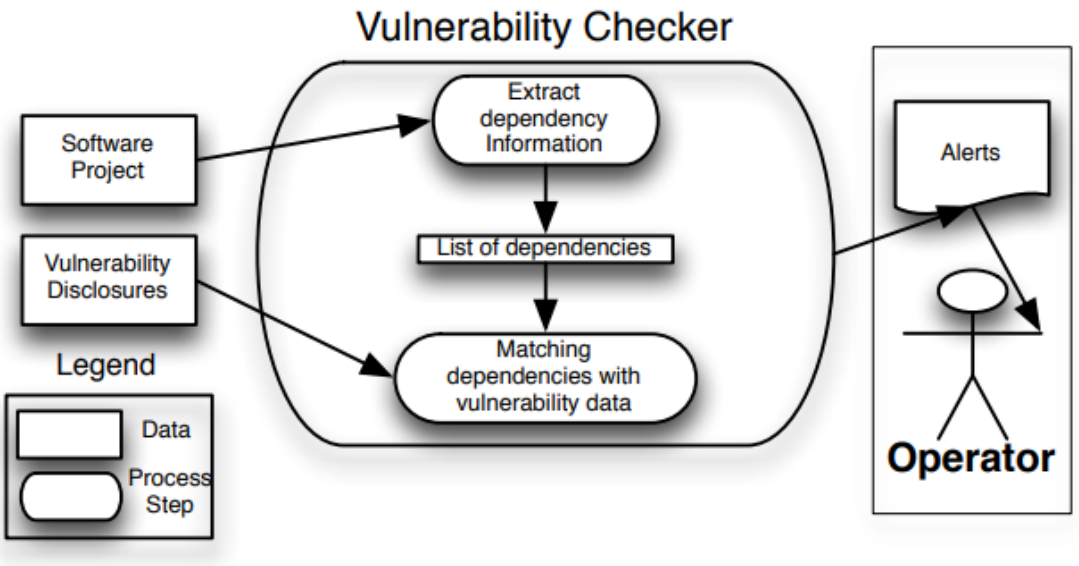


FIG. 2.1 : Processus de correspondance des vulnérabilités basée sur les noms [16]

Avantages

- Cette approche [16] est plus légère , avec moins de données à traiter et à stocker.
- Elle n'a pas besoin d'accéder au code source complet pour détecter les dépendances vulnérables, seul le fichier de configuration contenant la liste des dépendances déclarées est requis.
- Elle ne prend pas en compte les éléments spécifiques au langage de programmation.

Limites

- Cette approche [16] produit des faux positifs en signalant des vulnérabilités pour des dépendances déclarées dans le fichier de configuration mais jamais utilisées dans le code source de l'application.

2.2.2 Correspondance des vulnérabilités basée sur le code

L'approche [17] utilise l'Abstract Syntax Tree (AST), une représentation arborescente abstraite de la structure syntaxique d'un code source, où chaque nœud correspond à un élément syntaxique du code, tel qu'une expression, un opérateur, une variable, une méthode ou une classe. L'AST simplifie l'analyse du code en capturant les relations hiérarchiques entre ces éléments syntaxiques. [19]

L'approche [17] analyse les changements de code introduits par les commits de correction de vulnérabilités et compare les arbres de syntaxe abstraite (AST) des constructs (e.g., méthode,class) modifiées dans le fix commit d'une vulnérabilité i pour déterminer si

une dépendance i incluse dans une application a contient du code vulnérable à j comme montré dans la Figure 4.3.

L'analyse statique est utilisée pour déterminer si le code vulnérable est accessible, tandis que l'analyse dynamique vérifie si ce code a été exécuté, afin de déterminer si la vulnérabilité est exploitable.

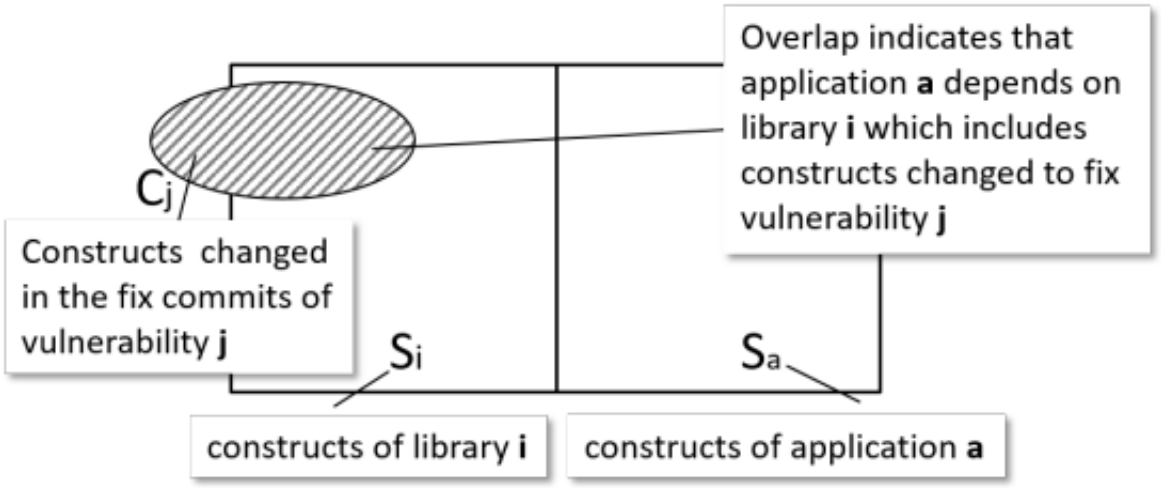


Fig. 2.2 : Détection des vulnérabilités en utilisant la correspondance des vulnérabilités basée sur le code [17]

Avantages

- En se basant sur le code source, cette approche [17] peut réduire les faux positifs.
- Cette approche permet une détection plus précise des vulnérabilités, car elle se base sur des preuves directes dans le code source .
- Cette méthode permet une compréhension plus approfondie de la nature des vulnérabilités.

Limites

- L'analyse du code source est plus complexe, ce qui rend le processus difficile à mettre en œuvre et nécessite beaucoup de temps et de ressources, ce qui peut le ralentir.
- Cette approche nécessite l'existence d'un commit de correction de la vulnérabilité afin de déterminer quelle partie du code est réellement vulnérable [17].

2.2.3 Correspondance des vulnérabilités basée sur le web sémantique

L'approche [18] vise à intégrer des ressources de connaissances hétérogènes, permettant une analyse efficace et une exploitation des données grâce à une représentation ontologique

unifiée. Cette représentation facilite le partage de connaissances de sécurité logicielle et les résultats d'analyse entre différentes ressources, permettant ainsi que les informations concernant les dépendances, leurs versions et les vulnérabilités associées soient centralisées et analysées de manière cohérente.

Les ontologies SECONT et MAVON[1] sont au cœur de cette méthodologie. SECONT est une représentation formelle des connaissances qui se concentre sur les vulnérabilités de sécurité, tandis que MAVON est dédiée à la gestion des dépendances des projets logiciels dans l'écosystème Maven[2] (un outil de gestion de projets et d'automatisation de la construction de logiciels Java). Ces ontologies sont accessibles au public, ce qui permet leur réutilisation par d'autres chercheurs et développeurs.

Comme montré dans la Figure 2.3, le processus commence par une extraction des informations sur les dépendances logiciels à partir des fichiers POM de Maven (le POM est un fichier XML utilisé par Apache Maven pour définir la structure d'un projet java) et des flux XML des vulnerabilities à partir de National Vulnerability Database (NVD) [4]. Les informations des dépendances logiciels et les vulnérabilités sont ensuite utilisées pour peupler les ontologies MAVON et SECONT et remplir les structures de données des ces ontologies via le cadre Jena (Jena est un framework Java pour la manipulation de données liées et le développement d'applications basées sur le Web sémantique), et les informations sont stockées dans un triple store pour faciliter les requêtes et le raisonnement.

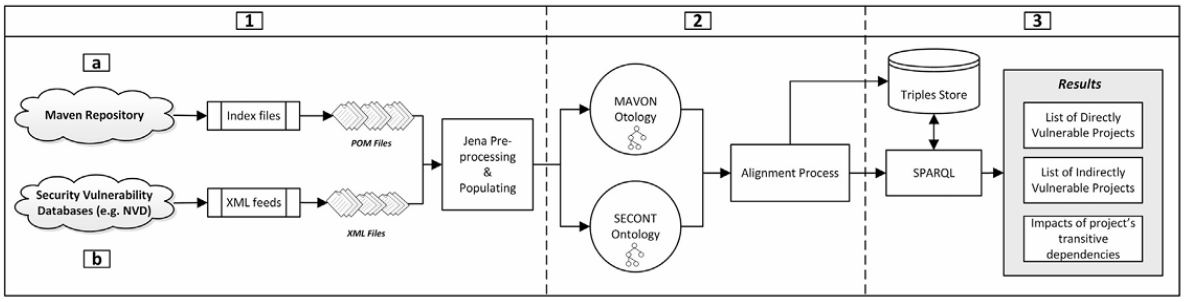


Fig. 2.3 : Processus de correspondance des vulnérabilités basée sur le web sémantique [18]

Une fois que les ontologies MAVON et SECONT sont peuplées avec des données sur les dépendances et les vulnérabilités, un processus de correspondance est lancé. Ce processus vise à aligner les différentes représentations ontologiques, c'est-à-dire à établir des liens entre des concepts similaires ou équivalents présents dans ces ontologies. Cela permet de réduire les différences sémantiques entre elles, facilitant ainsi l'intégration des connaissances.

Après cette étape d'alignement, il devient possible d'utiliser des requêtes SPARQL pour interroger la base de connaissances. Ces requêtes permettent d'extraire à la fois des connaissances explicites, c'est-à-dire directement présentes dans les données, et des connaissances implicites, qui peuvent être inférées grâce aux relations établies entre les

[1]SECONT et MAVON sont disponibles publiquement à l'adresse suivante : https://github.com/segps/segps-ontologies.

[2]http://search.maven.org/

ontologies. Par exemple, on peut récupérer des informations sur les vulnérabilités de sécurité associées à certaines dépendances, ainsi que des détails sur leurs impacts sur les projets logiciels. Cela enrichit l'analyse et aide à la prise de décisions concernant la sécurité des applications.

Avantages

- Cette approche [18] étend l'analyse traditionnelle des vulnérabilités au-delà des projets individuels pour inclure les dépendances transitives, offrant ainsi une portée d'analyse plus large.
- En créant une représentation unifiée et formelle des ressources de connaissances hétérogènes (SECONT et MAVON), cette approche facilite le partage et l'intégration des connaissances sur les vulnérabilités et les dépendances logicielles à travers différentes ressources.

Limites

- L'approche [18] peut générer des faux négatifs car les référentiels NVD et Maven peuvent utiliser des terminologies ou des conventions de nommage différentes pour des entités similaires.

Les approches d'identification des dépendances vulnérables incluent la correspondance basée sur les noms [16], qui détecte les vulnérabilités en comparant les noms des dépendances avec ceux des vulnérabilités connues, offrant une méthode légère mais sujette à des faux positifs. La correspondance basée sur le code [17] se distingue par l'analyse des changements introduits par les commits correctifs, permettant une détection plus précise des vulnérabilités en examinant directement les modifications apportées au code source. Cette approche identifie les vulnérabilités en comparant les arbres de syntaxe abstraite (AST) des portions de code modifiées, garantissant une meilleure contextualisation des vulnérabilités au sein de l'application. Bien que cette méthode soit plus précise, elle est également plus complexe et demandeuse en ressources. Enfin, la correspondance basée sur le web sémantique [18] utilise des ontologies pour intégrer et analyser de manière cohérente les données de vulnérabilités et de dépendances, facilitant une compréhension globale, mais pouvant générer des faux négatifs en raison des divergences terminologiques entre les référentiels.

2.3 Littérature sur la prédiction de la criticité des vulnérabilités

En raison de l'augmentation quotidienne des vulnérabilités publiées, les spécialistes en sécurité n'ont d'autre choix que de prioriser les plus critiques. Pour ce faire, plusieurs systèmes ont été mis en place, tels que le système CVSSv3 [3], pour évaluer la gravité des vulnérabilités de sécurité dans les systèmes informatiques.

Lorsqu'une nouvelle vulnérabilité est découverte, les chercheurs en sécurité prennent généralement plusieurs jours à quelques semaines pour publier ses métriques de gravité.

Cela souligne le besoin crucial d'un outil permettant d'extraire automatiquement ces métriques [20].

Plusieurs approches ont été proposées, incluant celles utilisant le machine learning, comme dans [21] et [22], celles basées sur le transfert learning en adaptant un modèle pré-entraîné à une nouvelle tâche en utilisant un ensemble de données spécifiques, telles que décrites dans [20] et [23], ainsi que d'autres qui utilisent le deep learning, comme présenté dans [24].

Comme le montre la Figure 2.4, ces approches [21] [22] [20] [23] [24] utilisent les descriptions des vulnérabilités rédigées en langage naturel comme jeu de données. Les données ont d'abord été collectées à partir de la base de données NVD [4]. Ensuite, le prétraitement du texte consiste à transformer les données textuelles brutes en un format plus propre et structuré afin de faciliter leur utilisation dans les modèles et les algorithmes. Puis, la représentation du texte consiste à convertir le texte en une forme numérique que les modèles peuvent traiter et analyser. Enfin, l'entraînement du modèle permet de prédire les métriques CVSSv3 [3] et de mettre en place un mécanisme d'évaluation du modèle proposé.

Les approches de prédiction de la criticité des vulnérabilités varient selon les techniques et algorithmes employés. La première approche [21] utilise les SVM et les forêts aléatoires pour classifier les scores CVSS [3] en dix classes, en se basant sur un dataset de 60 000 CVE. Cependant, la taille relativement limitée de ce dataset, y compris pour les données de test, peut affecter la robustesse des résultats, avec une accuracy de 87 % pour le SVM et de 88 % pour les forêts aléatoires.

La deuxième approche [22] repose sur la régression linéaire, qui traite chaque champ du CVSS comme une variable continue. Cette méthode obtient une accuracy globale de 82 %.

La troisième approche [20] utilise DistilBERT [25], obtenant une accuracy globale de 90 %. DistilBERT, en tant que modèle de langage avancé, est particulièrement efficace pour comprendre les nuances des descriptions textuelles des vulnérabilités. Cette compréhension approfondie du texte permet de prédire les champs du CVSS avec une grande précision, surpassant ainsi les autres approches.

La quatrième approche [23] s'appuie sur BERT [26] pour une modélisation multitâche combinant la régression et la classification pour les champs CVSS, avec une accuracy de 75 %. Enfin, la cinquième approche [24] utilise des réseaux de neurones convolutifs (CNN) pour une classification multi-classes des vulnérabilités, atteignant une accuracy de 81 %.

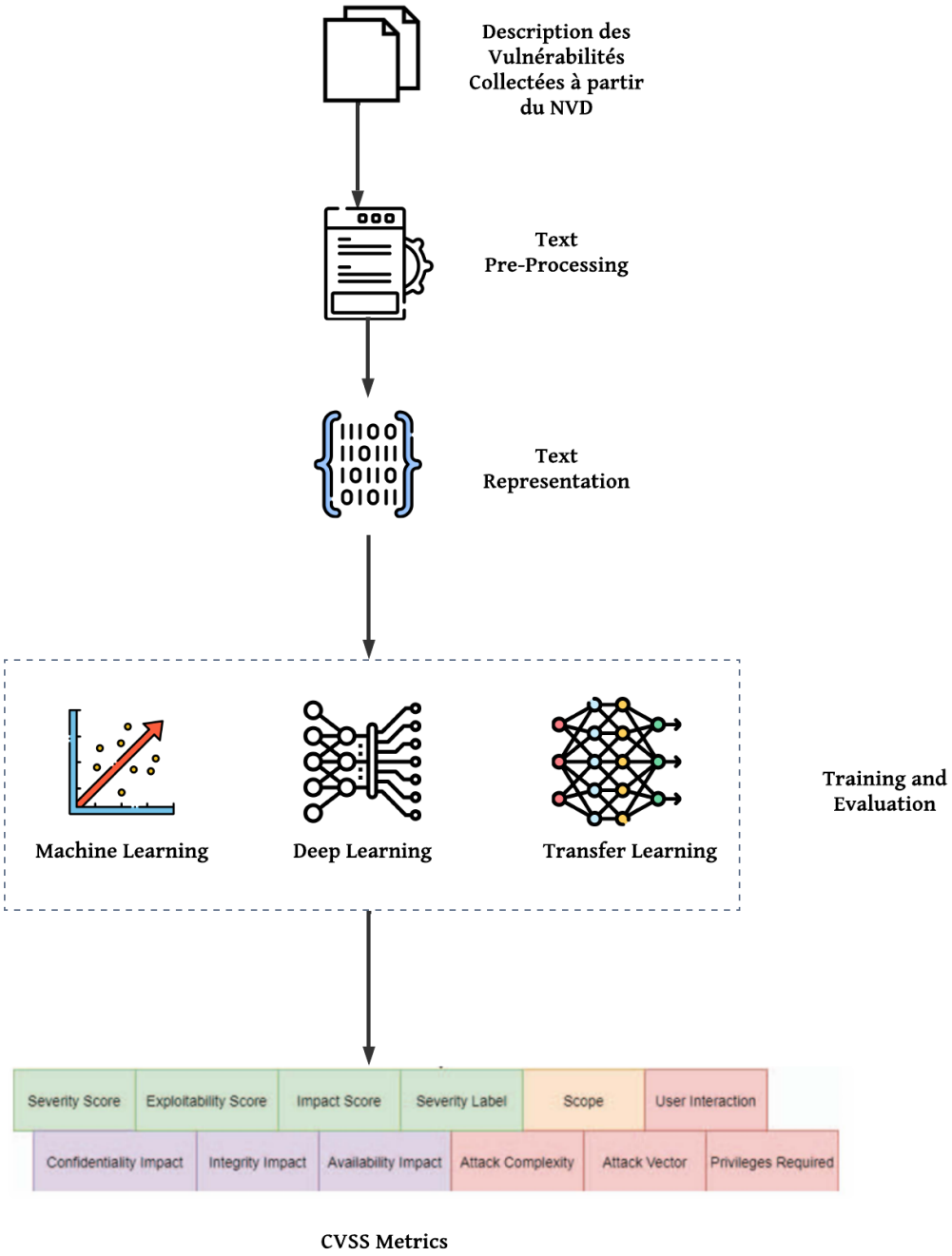


FIG. 2.4 : prédiction de la criticité des vulnérabilités

La Table 2.1 ci-dessous montre les principales différences entre les approches [21] [22] [20] [23] [24].

Approche	Algorithmes /Modèles clés	Méthodologie	Métriques de performance
[21]	SVM (Support Vector Machines) and Random forests	Les auteurs ont pris le score de gravité globale de CVSS, qui est un intervalle continu (de 0 à 10), et l'ont transformé en 10 classes distinctes pour résoudre un problème de classification.	une accuracy de 87 % pour le SVM et de 88 % pour Random forests
[22]	Linear regression	Cette approche a abordé la prédiction de la gravité CVSS [3] en traitant chaque champ individuel du CVSS comme un problème de régression linéaire.	accuracy globale (82 %)
[20]	distilBERT [25] (un modèle de langage basé sur l'architecture des transformers pré-entraîné sur de vastes corpus de texte non étiquetés)	Cette approche résout un problème de classification pour prédire les champs du CVSS 3 des vulnérabilités en se basant sur leurs descriptions, en utilisant DistilBERT [25].	accuracy globale (90 %)
[23]	BERT [26] (un transformeur bidirectionnel pré-entraîné utilisant une combinaison de modélisation du langage masqué et de prédiction de la phrase suivante sur un large corpus de Wikipédia.)	Cette approche détermine les champs CVSSv3 en s'appuyant sur un modèle d'apprentissage multitâche avec partage de paramètres (des tâches de régression pour les champs quantitatifs et des tâches de classification pour les champs qualitatifs).	accuracy globale (75 %)
[24]	CNN (Convolutional Neural Network)	Cette approche a abordé la tâche de prédiction de la classe de gravité CVSS [3] comme un problème de classification de texte multi-classes, en s'appuyant sur des réseaux de neurones convolutifs (CNN).	accuracy (81 %)

Tab. 2.1 : Approches de prédiction de la criticité des vulnérabilités

2.4 Conclusion

En résumé, ce chapitre a examiné différentes approches pour l'identification des vulnérabilités dans les dépendances logicielles, chacune présentant ses propres avantages et limites. Nous avons analysé ces approches sous divers aspects, notamment leurs objectifs, leurs entrées, leur processus d'identification et leurs résultats. L'approche basée sur l'analyse de code [17] s'est avérée la plus précise, offrant une meilleure contextualisation des dépendances vulnérables et permettant une détection précise de celles-ci.

Nous avons également exploré les techniques de prédiction automatique de la criticité des vulnérabilités, qui permettent de prioriser les vulnérabilités détectées. Des modèles préentraînés en traitement du langage naturel (NLP), comme DistilBERT [25], ont montré leur efficacité dans la prédiction de la criticité des vulnérabilités, en utilisant les descriptions textuelles pour obtenir des résultats précis. Cette diversité d'approches nous a fourni des perspectives précieuses pour le développement de notre propre solution. Dans le chapitre suivant, nous présenterons les besoins de notre projet ainsi que la méthodologie de développement suivie pour le réaliser.

Chapitre 3

Méthodologie de Développement et Analyse de Besoins

3.1 Introduction

Le but de notre travail est de développer un outil informatique permettant de contextualiser les dépendances vulnérables dans une application donnée. Ce processus consiste à déterminer si une vulnérabilité (CVE) associée à une dépendance logicielle est réellement exploitable dans le contexte spécifique de l'application analysée. Cela implique non seulement d'analyser la présence de la dépendance vulnérable, mais aussi d'examiner son utilisation concrète dans le code, grâce à une analyse statique du code source de l'application. Le processus inclut également la priorisation des vulnérabilités détectées afin que les développeurs puissent se concentrer sur les vulnérabilités les plus exploitables.

Dans ce chapitre, nous exposons d'abord la méthodologie adoptée pour l'analyse, la conception, et la réalisation de notre système. Nous abordons ensuite la première étape de notre démarche, qui consiste en l'analyse des besoins. Cette analyse vise à identifier les acteurs impliqués dans notre système, à recenser les besoins fonctionnels et non fonctionnels (techniques), ainsi qu'à établir le diagramme de cas d'utilisation en UML. Enfin, nous développons le diagramme d'activité, qui décrit le processus de l'approche proposée.

3.2 Méthodologie de développement

3.2.1 Présentation de la méthodologie choisie

Pour notre projet, nous avons choisi dès le départ une méthode agile, un cadre qui englobe plusieurs approches basées sur un développement itératif et incrémental. Parmi les méthodes les plus couramment utilisées figurent XP (Extreme Programming)[1] et Scrum[2].

Dans notre cas, nous avons opté pour Scrum, qui est une approche empirique où les décisions sont fondées sur l'observation et l'expérimentation. Ce cadre repose sur trois principes essentiels : la transparence, qui assure que tous les aspects du processus sont visibles à ceux qui en sont responsables, l'inspection, qui permet de détecter les écarts indésirables dans le développement grâce à des revues régulières, et l'adaptation, qui encourage des ajustements rapides en fonction des retours et des observations. Ensemble, ces principes favorisent un mode de travail itératif, garantissant l'amélioration continue et la réactivité face aux changements. [27]

Scrum fonctionne avec les les éléments suivants :

- Les Sprints, qui durent généralement un mois, permettent de livrer des incréments de travail apportant de la valeur. Tout au long de chaque Sprint, des retours continus sont collectés pour évaluer et ajuster à la fois le travail effectué et le processus en cours. [27]
- L'équipe Scrum se compose d'un Scrum Master, d'un Product Owner et de Développeurs, qui sont responsables de transformer les tâches en un incrément de valeur au cours des Sprints. [27]

[1] http://www.extremeprogramming.org/
[2] https://www.scrum.org/learning-series/what-is-scrum/

- À la fin de chaque Sprint, les résultats sont examinés par l'équipe Scrum et les parties prenantes pour améliorer le processus pour les prochains sprints. [27]

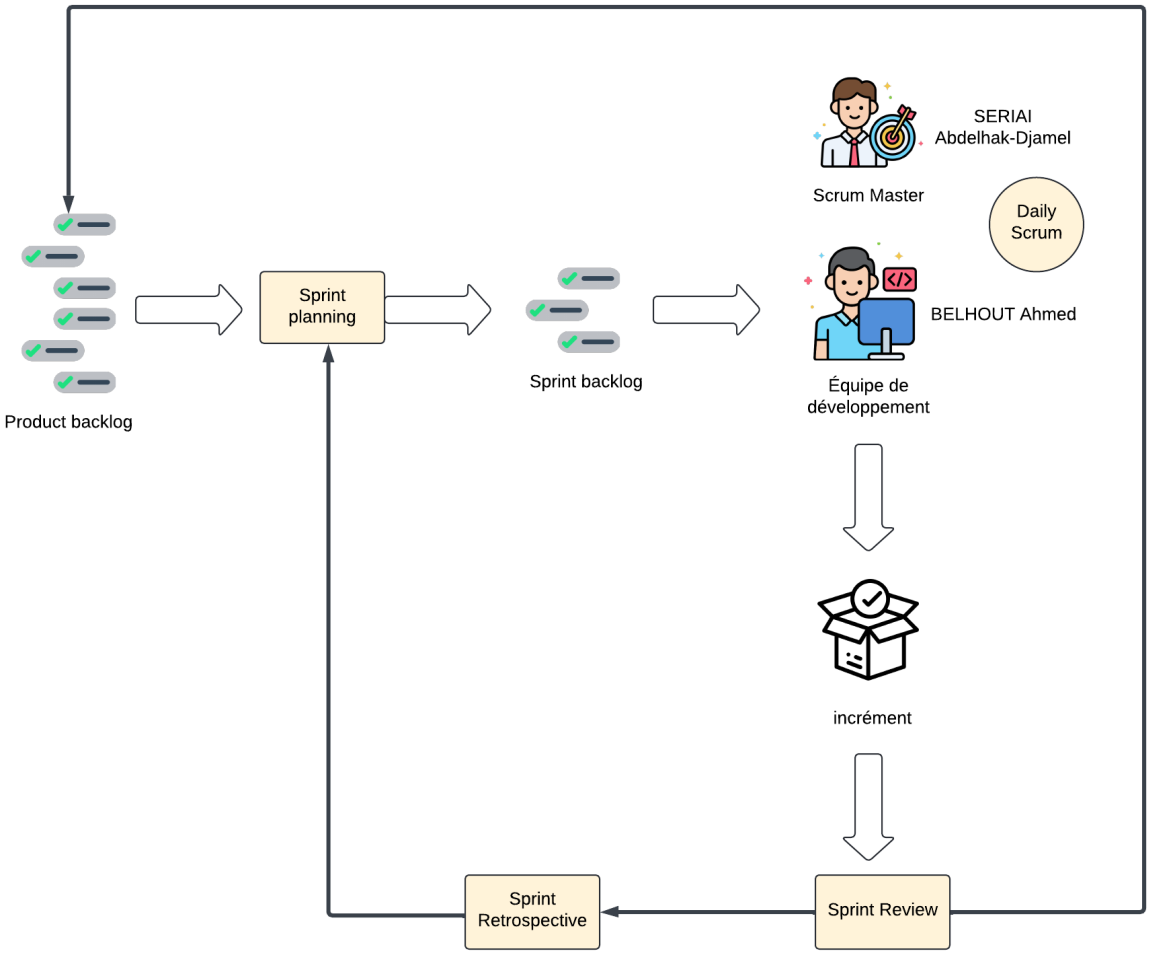


Fig. 3.1 : Méthode Scrum [27]

Dans notre cas, les rôles de Scrum sont répartis comme suit :

- Le Scrum Master, qui est le facilitateur de l'équipe Scrum, s'assure que Scrum est compris et appliqué correctement par l'équipe. Dans notre cas, il s'agit de M. SERIAI Abdelhak-Djamel, mon promoteur de stage au sein du laboratoire LIRMM.
- Le Product Owner, qui représente le client et qui est responsable de maximiser la valeur du produit résultant du travail de l'équipe de développement, est également M. SERIAI Abdelhak-Djamel dans notre cas.
- L'équipe de développement effectue le travail nécessaire pour livrer un incrément de produit fonctionnel à la fin de chaque Sprint. Dans notre cas, c'est moi-même qui constitue cette équipe.

La figure suivante illustre les tableaux des tâches que nous utilisons pour gérer les tâches au cours de nos sprints :

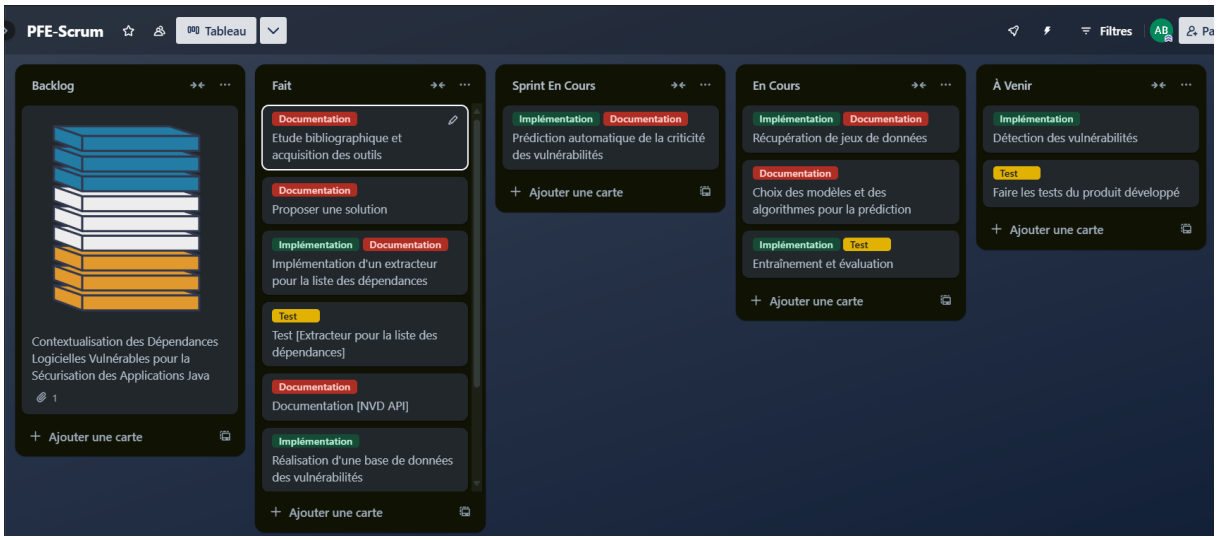


Fig. 3.2 : Tableaux des tâches

3.2.2 Motivation du choix de la méthode Scrum

Nous avons choisi cette méthode agile car elle présente plusieurs avantages pour le développement de projets petits et moyens, et qui sont : [27]

- **Flexibilité pour s'adapter aux changements :** Scrum permet d'intégrer facilement les changements et les nouvelles priorités en cours de projet. Les itérations courtes et les revues régulières permettent à l'équipe de réagir rapidement aux retours des parties prenantes et aux évolutions du marché.

- **Livraisons fréquentes de produits fonctionnels :** À la fin de chaque Sprint, une version fonctionnelle du produit, ou incrément, est livrée. Cela garantit que le travail apporte de la valeur rapidement et que les utilisateurs peuvent tester des fonctionnalités au fur et à mesure de leur développement.

- **Amélioration continue grâce aux rétrospectives :** Après chaque Sprint, l'équipe tient une rétrospective pour évaluer ce qui a bien fonctionné et ce qui pourrait être amélioré. Cela encourage une culture d'apprentissage constant et d'optimisation des processus de travail.

- **Transparence totale sur l'avancement du projet :** Scrum met en avant la transparence en utilisant des outils tels que le tableau des tâches et les revues de Sprint. Tout le monde peut voir l'état d'avancement des travaux, les obstacles rencontrés, et les progrès réalisés.

- **Communication quotidienne pour maintenir l'alignement :** Les réunions quotidiennes de type "Daily Scrum", permettent à l'équipe de synchroniser ses efforts et de partager les mises à jour et de discuter des problèmes éventuels.

3.3 Analyse de besoins

L'identification et l'analyse des besoins est une étape cruciale lors du développement d'un outil informatique, car elle permet de bien cerner les exigences et les attentes liées à cet outil. Dans le cadre de notre projet, l'objectif est la contextualisation des dépendances vulnérables, c'est-à-dire l'identification des dépendances vulnérables à l'aide de l'analyse statique de code source, basée sur les commits correctifs des vulnérabilités, et la priorisation des vulnérabilités détectées en fonction de leur criticité.

3.3.1 Identification des acteurs du système

Notre système est destiné aux développeurs qui souhaitent analyser leurs projets et leurs dépendances logicielles pour détecter et prioriser les vulnérabilités associées à ces dépendances.

3.3.2 Spécifications du système

Pour répondre aux besoins de notre client, nous avons élaboré les spécifications fonctionnelles et techniques. Concernant les spécifications fonctionnelles, nous assignons à chacune une priorité selon la méthode MoSCoW :

- M (Must Have) : réfère une spécification indispensable.
- S (Should Have) : réfère à spécification importante mais pas cruciale.
- C (Could Have) : réfère à spécification souhaitable mais pas nécessaire.
- W (Would Have) : réfère à spécification qui pourrait être dans une dernière version.

Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent les fonctionnalités et les comportements attendus de notre système.

ID	Spécification	Priorité
SF01	Le système doit permettre aux développeurs de sélectionner le projet en entrée.	M
SF02	Le système doit permettre aux développeurs de lancer le processus de contextualisation des dépendances vulnérables.	M
SF03	Le système doit effectuer une analyse statique du code pour identifier l'utilisation des dépendances vulnérables.	M
SF04	Le système doit surveiller le comportement d'exécution de l'application pour identifier les vulnérabilités potentielles exploitables en temps réel.	C
SF05	Le système doit permettre aux développeurs d'extraire la liste des dépendances de leur projet.	S
SF06	Le système doit analyser les références des CVE pour rechercher leurs commits correctifs.	M
SF07	Le système doit mettre en place une technique de priorisation des vulnérabilités.	M
SF08	Le système doit permettre aux développeurs de visualiser les dépendances de leur projet.	S
SF09	Le système doit permettre aux développeurs de visualiser les résultats de contextualisation des dépendances vulnérables.	M

Tab. 3.1 : Spécifications fonctionnelles du système

Spécifications techniques

ID	Spécification
ST01	Le système doit offrir une base de données des vulnérabilités qui est régulièrement actualisée pour fournir des informations à jour.
ST02	Le système doit utiliser une base de données relationnelle des vulnérabilités.
ST03	Le système doit prendre en entrée une application Java sous la forme d'un répertoire de fichiers.
ST04	Le système doit fournir une interface utilisateur conviviale permettant de visualiser les résultats des analyses.

Tab. 3.2 : Spécifications techniques du système

3.3.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est une notation qui permet de modéliser les différentes manières dont les acteurs interagissent avec un système. La Figure 3.3 ci-dessous illustre le diagramme de cas d'utilisation de notre système et comment le développeur

interagit avec lui.

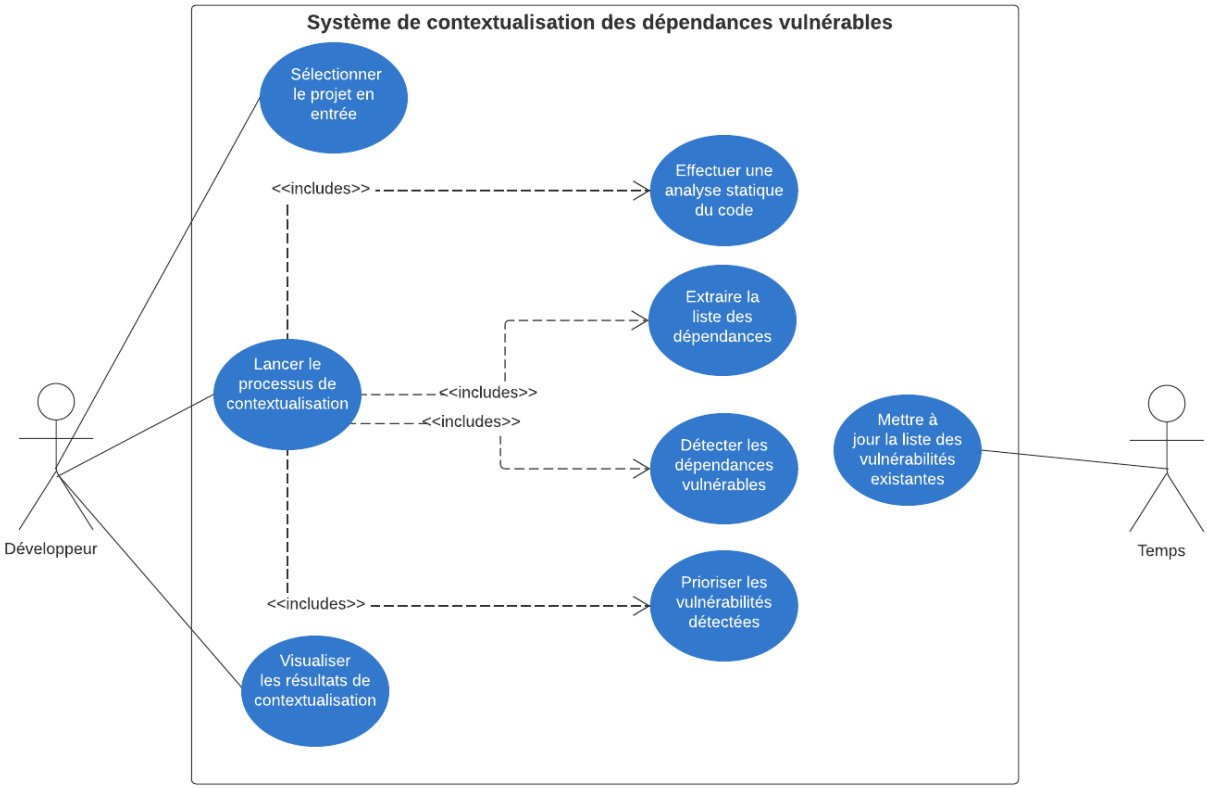


Fig. 3.3 : Diagramme de cas d'utilisation du système

Documentation des cas d'utilisation

Nous avons choisi de documenter les deux cas d'utilisation **Lancer le processus de contextualisation** et **Visualiser les résultats de contextualisation** car ils couvrent les aspects essentiels du fonctionnement de l'outil. Le premier cas d'utilisation est crucial pour la contextualisation des dépendances vulnérables, tandis que le second est indispensable pour afficher les résultats de manière claire et exploitable. En documentant ces cas, nous assurons une compréhension complète et une utilisation efficace du système pour les développeurs.

CU : Lancer le processus de contextualisation
ID : 1
Description : Lancer le processus d'analyse et de contextualisation des dépendances vulnérables du projet sélectionné.
Acteurs primaires : Développeur
Acteurs secondaires : Aucun
Précondition : Un projet doit être sélectionné.
Enchainement principal : 1. Le CU démarre lorsque le développeur clique sur "Démarrer l'analyse" 2. include (Extraire la liste des dépendances du projet) 3. Le système obtient la liste des dépendances du projet 4. include (Effectuer une analyse statique du code) 5. Le système obtient l'AST du projet. 6. include (Détecter les dépendances vulnérables) 7. include (Prioriser les vulnérabilités détectées) 8. Le système affiche les résultats de contextualisation
Postcondition :
Enchainement principal :

Tab. 3.3 : Documentation de CU "Lancer le processus de contextualisation"

CU : Visualiser les résultats de contextualisation
ID : 2
Description : Visualiser les résultats de contextualisation des dépendances vulnérables
Acteurs primaires : Développeur
Acteurs secondaires : Aucun
Précondition : Le développeur doit déclencher le cas d'utilisation de lancement du processus de contextualisation.
Enchainement principal : 1. Le CU démarre lorsque le développeur clique sur "Visualiser les résultats" 2. Le système affiche les résultats de contextualisation.
Postcondition :
Enchainement principal :

Tab. 3.4 : Documentation de CU "Visualiser les résultats de contextualisation"

3.3.4 Diagramme d'activité

La figure 3.4 ci-dessous illustre le diagramme d'activité du processus d'analyse des dépendances vulnérables dans un projet donné.

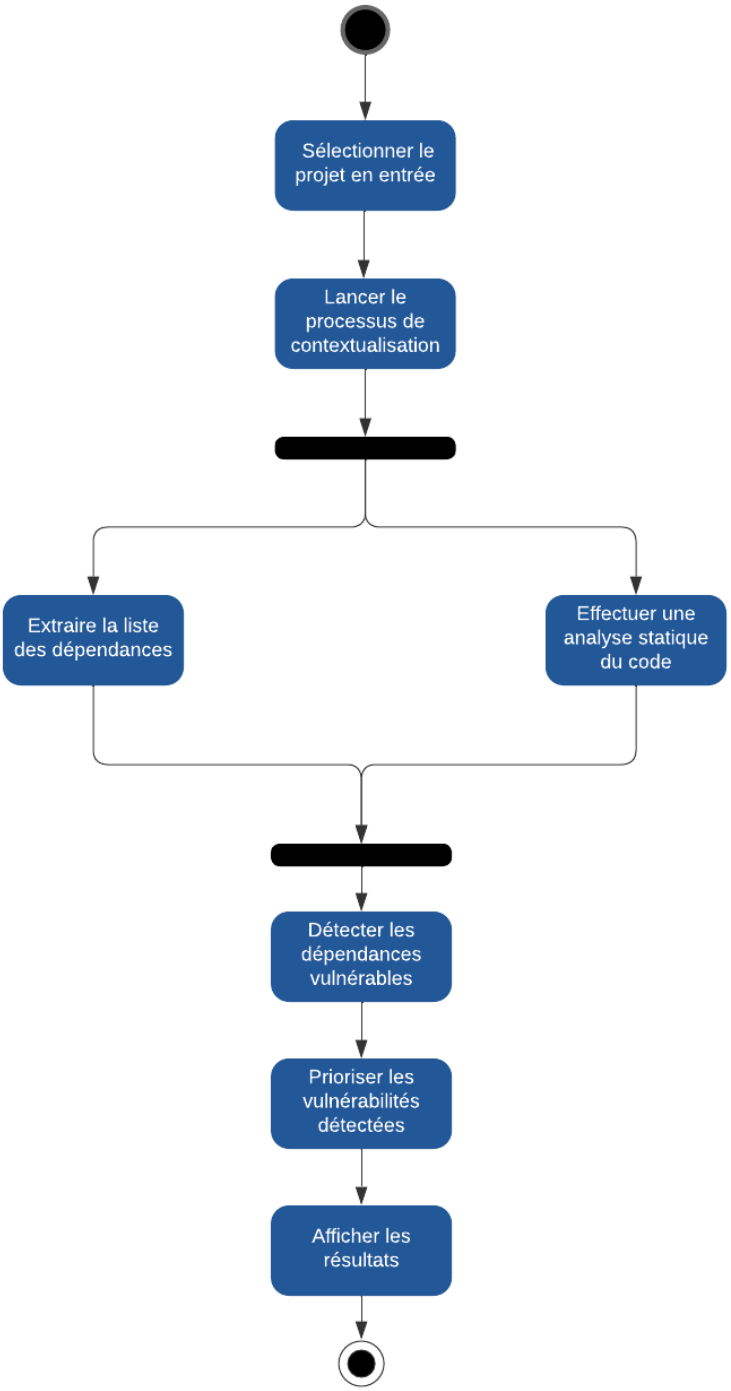


FIG. 3.4 : Diagramme d'activité du système

Le processus commence par la sélection du projet en entrée. Une fois le projet sélectionné, le processus de contextualisation est lancé. Le système extrait ensuite la liste des

dépendances du projet et effectue une analyse statique du code pour identifier les dépendances vulnérables. Ces dépendances et les résultats de l'analyse statique sont ensuite comparés avec la base de données des vulnérabilités du système pour déterminer celles qui sont vulnérables. Ensuite, ces vulnérabilités détectées sont priorisées en fonction de leur criticité. Enfin, les résultats de la contextualisation sont présentés au développeur.

3.4 Conclusion

Nous avons adopté la méthode agile Scrum parce que l'un des avantages de cette approche est qu'elle nous permet de développer les fonctionnalités recensées de manière incrémentale et itérative. De plus, nous avons également utilisé le langage UML pour modéliser notre système à travers l'étape d'analyse, ce qui nous a permis d'identifier clairement les fonctionnalités que notre client souhaitait réaliser. Dans le chapitre qui suit, nous poursuivrons notre processus de développement en présentant la conception du système.

Chapitre 4

Conception

4.1 Introduction

Après avoir identifié les besoins et les exigences du système que nous souhaitons réaliser, ce chapitre se concentre sur l'étape cruciale du processus de développement, à savoir la conception de notre solution. Nous commençons par une vue d'ensemble du système à développer avec un schéma qui décrit tout le processus. Ensuite, chaque composant est détaillé, tout en donnant l'algorithme proposé pour sa mise en œuvre, garantissant ainsi une compréhension complète et une intégration cohérente de la solution.

4.2 Vue globale de la solution

L'outil à développer recevra en entrée le code source d'un projet Java sous forme de dossier et produira en sortie un rapport de contextualisation des dépendances vulnérables.

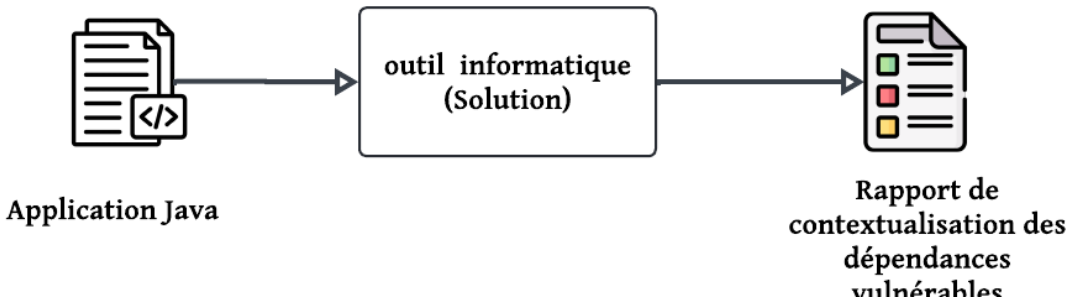


Fig. 4.1 : Paramètres d'Entrée et Résultats de Sortie de l'Outil à développer

D'après l'analyse élaborée dans le chapitre précédent, nous avons recensé les fonctionnalités suivantes :

- **Construction de la base de données des vulnérabilités**
  Tout d'abord, il est nécessaire de construire une base de données des vulnérabilités. Cette base servira de source d'informations pour identifier les vulnérabilités et les priorités. Elle devra être capable de répondre à plusieurs requêtes, telles que les vulnérabilités associées à une dépendance spécifique, le commit correctif de vulnérabilité, ainsi que le niveau de criticité des vulnérabilités.

- **Prédiction de la criticité des vulnérabilités**
  Lorsqu'une vulnérabilité est publiée dans la base de données NVD [4], elle ne comporte pas immédiatement les métriques de criticité. Ces métriques sont élaborées par des experts en sécurité, un processus qui peut prendre plusieurs jours, voire des mois. D'où la nécessité de prédire ces métriques automatiquement afin de prioriser les vulnérabilités de la plus critique à la moins critique.

- **Extraction de la liste des dépendances**
  Dans l'aspect d'analyse du code, il est nécessaire d'identifier la liste des dépendances (packages) utilisées dans le projet Java à analyser. Ces dépendances sont

déclarées dans le fichier de configuration pom.xml. Cette liste sera ensuite utilisée pour analyser et détecter les dépendances vulnérables.

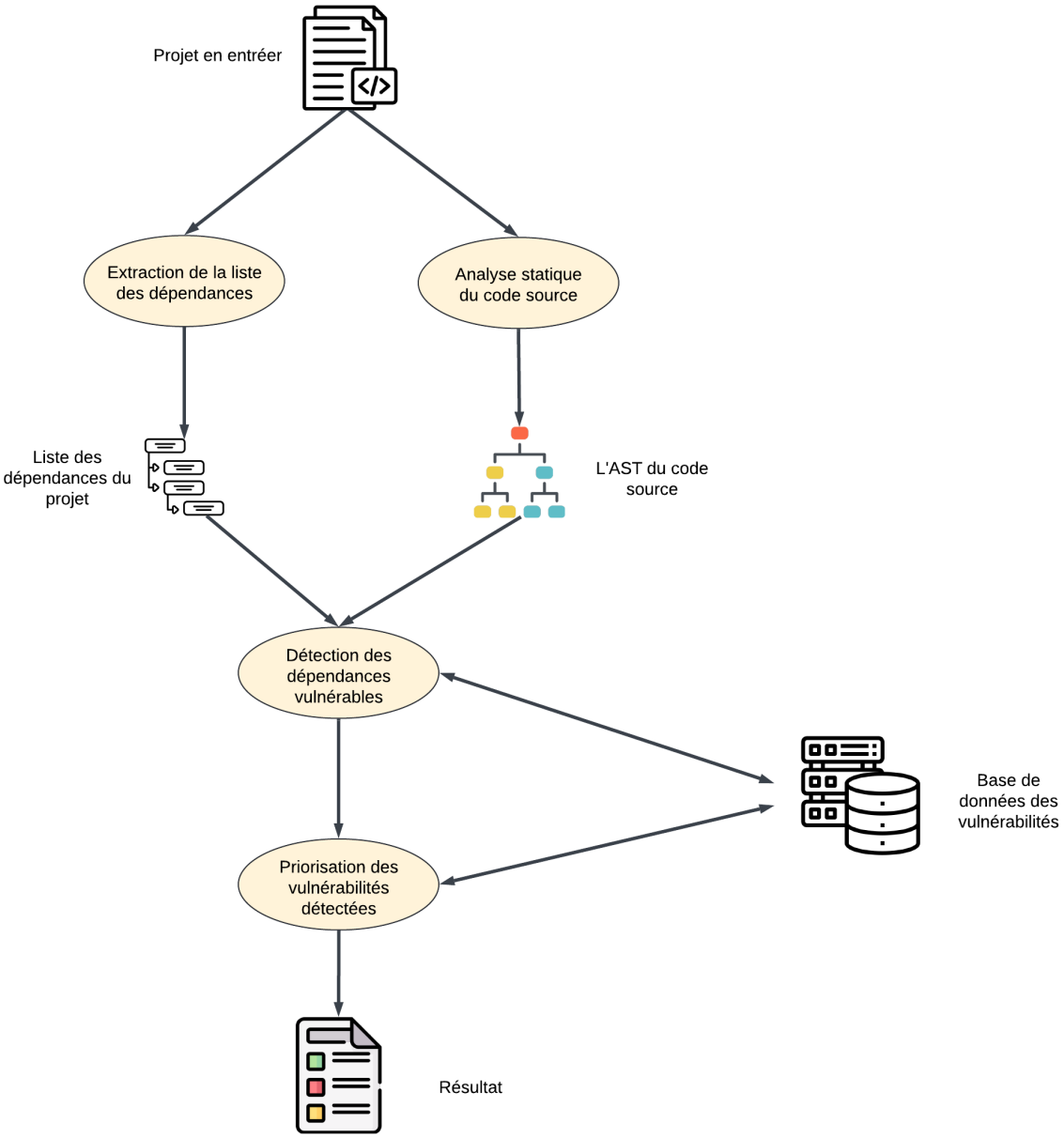


Fig. 4.2 : Vue globale de la solution

- **Analyse statique du code source**

  L'analyse statique du code sera utilisée pour extraire l'AST (Abstract Syntax Tree) du projet, qui décrit les éléments du code source à analyser. Cette étape est cruciale pour identifier les structures du code avant de procéder à la détection des vulnérabilités.

- **Détection des dépendances vulnérables**
  Maintenant que la base de données des vulnérabilités est prête, que la liste des

dépendances a été extraite, et que l'AST du code source du projet à analyser a été générée, il reste à mettre en place les techniques de détection des dépendances vulnérables, qui seront détaillées dans la conception détaillée.

- **Priorisation des vulnérabilités détectées**

  Après la détection des dépendances vulnérables, il est essentiel de prioriser ces vulnérabilités en fonction de leurs métriques CVSS. Cela permet aux développeurs de concentrer leurs efforts sur les vulnérabilités les plus critiques.

La figure 4.2 montre une vue d'ensemble de la solution. Le processus débute par la sélection du projet à analyser. Une fois le projet choisi, le système extrait alors la liste des dépendances du projet et réalise une analyse statique du code pour identifier les dépendances vulnérables. Les dépendances détectées et les résultats de l'analyse sont ensuite comparés à la base de données des vulnérabilités du système pour déterminer celles qui sont réellement vulnérables. Les vulnérabilités identifiées sont alors classées selon leur criticité. Enfin, les résultats de la contextualisation sont présentés au développeur.

4.3 Conception détaillée

Cette section détaille les étapes de l'approche proposée, en offrant une vue d'ensemble complète de chaque phase du processus. Tout d'abord, nous décrivons la manière dont les données sont collectées et préparées, suivie d'une explication des techniques utilisées pour prioriser les vulnérabilités identifiées en fonction de leur criticité. Ensuite, nous abordons les méthodes employées pour identifier les vulnérabilités associées au projet à analyser. Chaque étape est essentielle pour le processus de contextualisation des dépendances vulnérables.

4.3.1 Construction de la base de données des vulnérabilités

Les bases de données publiques des vulnérabilités des dépendances logicielles présentent plusieurs limites, telles que le manque de métriques de criticité des vulnérabilités, l'absence de correctifs pour certaines vulnérabilités, ainsi que des problèmes de disponibilité de ces bases. Il est donc indispensable de constituer notre propre base de données des vulnérabilités, afin de dépasser ces limitations.

Présentation de la source de données

Pour récupérer les données sur les vulnérabilités des dépendances logicielles, nous avons utilisé la base de données NVD (National Vulnerability Database) [4], qui est une ressource centralisée et publique maintenue par le gouvernement des États-Unis. La NVD fournit des informations détaillées sur les vulnérabilités découvertes, incluant des descriptions, des identifiants de vulnérabilités, des références aux solutions correctives, ainsi que des métriques de criticité des vulnérabilités.

La NVD propose une API pour récupérer les données sur les vulnérabilités via le point de terminaison (endpoint) suivant :



Fig. 4.3 : Le point de terminaison pour récupérer les données de la NVD [4]

Voir l'annexe A 5.9 pour un exemple de contenu d'une vulnérabilité récupérée de la NVD.

La NVD [4] sera utilisée comme source de vulnérabilités pour remplir notre base de données, qui sera schématisée dans la section suivante.

Structure de la base de données

Le schéma de notre base de données des vulnérabilités (voir Figure 4.4) est structuré pour fournir une vue détaillée et interconnectée des vulnérabilités. Chaque vulnérabilité est représentée par un enregistrement dans la table CVE, qui constitue le cœur de notre base de données. Cette table est reliée à plusieurs autres tables, chacune apportant des informations supplémentaires sur la vulnérabilité.

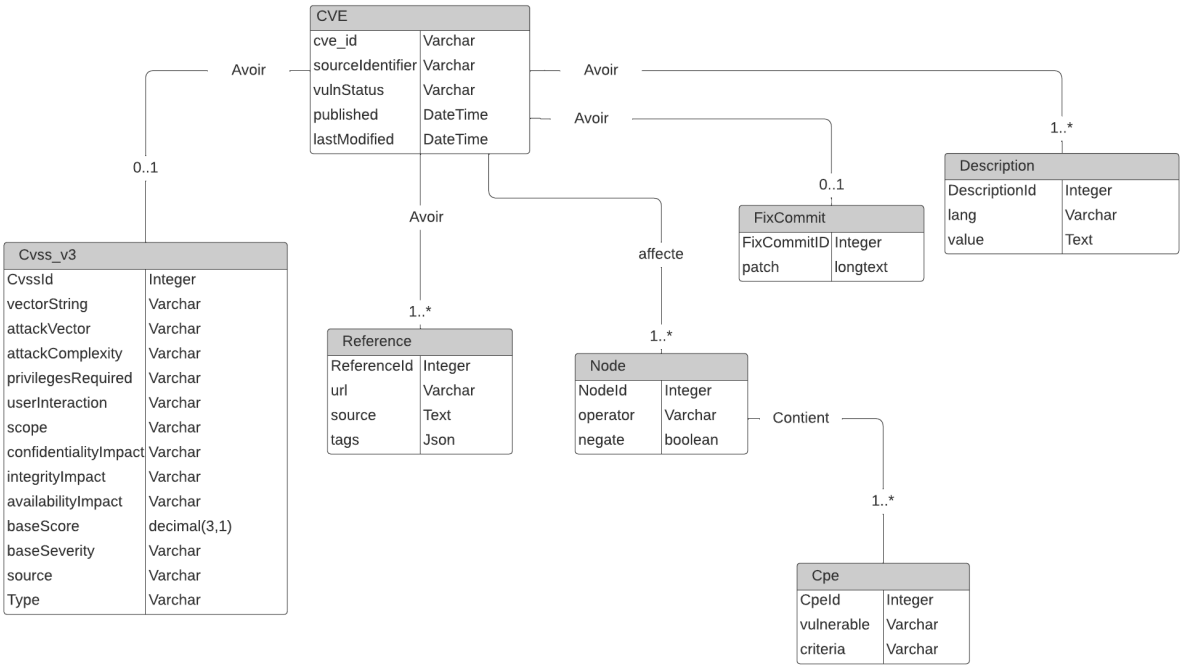


Fig. 4.4 : Schéma de notre base de données

Tout d'abord, chaque entrée dans la table CVE peut être associée à plusieurs en-

registrements dans la table Références, ce qui permet de lier la vulnérabilité à diverses sources d'information telles que des conseils de sécurité, des rapports d'exploit ou des commits correctifs. De même, une vulnérabilité peut avoir plusieurs descriptions dans la table Description, offrant ainsi différentes perspectives ou traductions de la vulnérabilité.

En ce qui concerne l'évaluation de la gravité, chaque enregistrement de la table CVE est associé à une unique métrique CVSS [3] version 3 dans la table CVSSV3. Cette relation permet d'obtenir une évaluation précise du niveau de sévérité de la vulnérabilité. Parmi les champs CVSS, *baseScore* mesure l'impact fondamental de la vulnérabilité sur un système, *attackVector* décrit le contexte dans lequel l'attaque peut être initiée, et *confidentialityImpact* quantifie les dommages potentiels sur la confidentialité des ressources.

Les vulnérabilités affectent également plusieurs nœuds, qui précisent les éléments spécifiques concernés. Chaque nœud dans la table Node peut contenir plusieurs entrées CPE dans la table CPE. Ces entrées CPE identifient avec précision les produits logiciels ou matériels touchés par la vulnérabilité en tenant compte du champ *criteria*.

Toutes les tables seront remplies à partir de la NVD [4], à l'exception de la table **FixCommit**, qui est destinée à contenir les modifications du code source abordées dans le commit correctif de la vulnérabilité. Cette table sera remplie d'une autre manière, expliquée dans la section suivante.

Identification des commits correctifs des vulnérabilités

Les commits correctifs des vulnérabilités désignent les modifications apportées au code source d'un logiciel pour corriger des failles de sécurité identifiées. Ces commits permettent d'identifier la partie du code réellement vulnérable dans une vulnérabilité donnée. Malheureusement, la base de données NVD [4] ne contient pas cette information, d'où la nécessité d'identifier les commits correctifs, s'ils existent, en exploitant les références associées à la vulnérabilité.

Tout d'abord, comme illustré dans la Figure 4.5, nous parcourons toutes les vulnérabilités de notre base de données et, pour chaque vulnérabilité, nous extrayons la liste des références associées. Ensuite, nous filtrons cette liste afin d'identifier les références qui pointent vers un commit sur GitHub et qui portent le tag Patch, indiquant que ce commit est un commit correctif.

S'il y a une référence qui pointe vers un commit correctif sur GitHub, nous procédons alors à une requête GET vers l'API GitHub pour récupérer les modifications de code (lignes de code ajoutées et lignes de code supprimées) abordées par le commit en question. Ensuite, nous insérons les modifications de code dans le champ patch de la table FixCommit de notre base de données en associant cette entrée avec la vulnérabilité correspondante.

S'il n'existe pas de référence pointant vers un commit correctif, nous passons à la vulnérabilité suivante pour l'analyser.

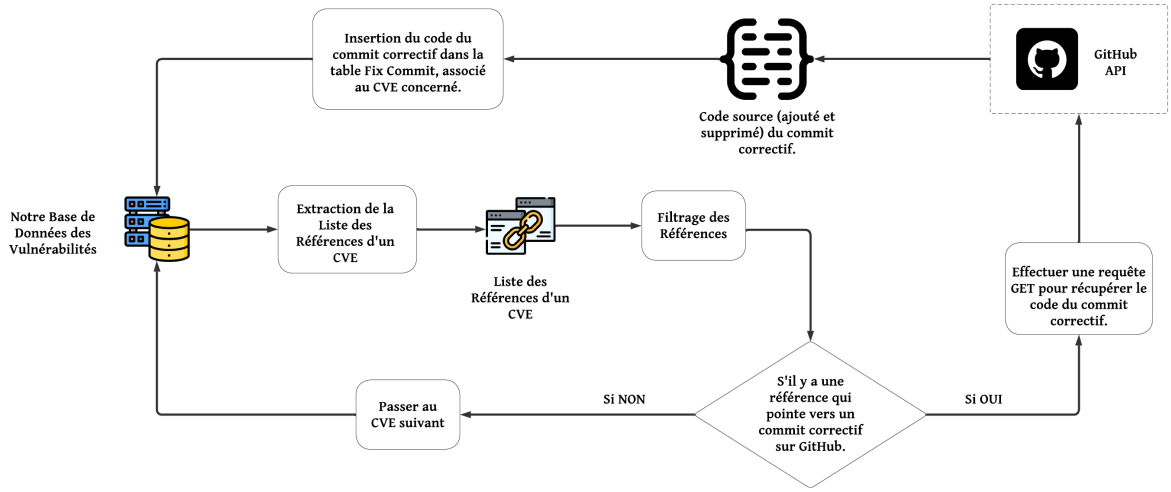


FIG. 4.5 : Processus d'identification des commits correctifs des vulnérabilités

Maintenance de la base de données

Pour garantir que notre base de données reste actuelle et pertinente, il est crucial de planifier et d'exécuter des mises à jour régulières afin d'intégrer les nouvelles données de la NVD [4] ainsi que les informations mises à jour. Il est donc recommandé d'établir un calendrier de mises à jour pour automatiser ce processus autant que possible, réduisant ainsi les risques d'erreurs manuelles et assurant une intégration fluide des nouvelles informations. La mise en place de processus automatisés se fait par l'exécution de scripts d'extraction et de chargement de données à partir de la NVD, grâce au point de terminaison offert par la NVD qui permet de récupérer les vulnérabilités modifiées, ajoutées et supprimées depuis une date donnée. Notre script sera exécuté chaque jour pour récupérer les vulnérabilités modifiées depuis le jour précédent et les charger dans notre base de données. Ce processus nous assure une gestion cohérente et continue des données tout en minimisant les interruptions et les risques potentiels.

La NVD propose un point de terminaison (voir Figure 4.6) permettant de filtrer et de renvoyer uniquement les CVEs qui ont été modifiés pour la dernière fois pendant une période spécifique définie par deux dates : lastModStartDate, qui spécifie la date de début de la période, et lastModEndDate, qui spécifie la date de fin de la période.

Request all CVE change histories between the start and end datetimes

```
https://services.nvd.nist.gov/rest/json/cvehistory/2.0/?
changeStartDate=2021-08-04T13:00:00.000%2B01:00&changeEndDate=2021-10-
22T13:36:00.000%2B01:00
```

FIG. 4.6 : Le point de terminaison pour filtrer et renvoyer uniquement les CVEs modifiés [4]

4.3.2 Prédiction de la criticité des vulnérabilités

Lorsqu'une vulnérabilité est publiée dans la base de données NVD [4]nvd, elle contient uniquement une description et des références. Les métriques de criticité sont ajoutées à cette vulnérabilité après quelques semaines, voire quelques mois, par des spécialistes en sécurité. D'où la nécessité de mettre en place des techniques permettant de prédire ces métriques automatiquement, ce qui réduit le temps et le coût.

Notre solution vise à intégrer l'apprentissage par transformeurs, en particulier les algorithmes de traitement automatique du langage (NLP), afin de classer automatiquement les descriptions de CVE et de prédire la catégorie de chaque métrique CVSS [3]. Par exemple, une métrique telle que l'Impact de la Confidentialité (Confidentiality Impact) peut être classée en trois catégories : None (Aucune), Low (Faible), ou High (Élevée). Cette classification aide à évaluer l'impact potentiel d'une vulnérabilité sur la confidentialité des données.

Nous avons choisi d'utiliser le modèle préentraîné en NLP DistilBERT [25], une version améliorée et optimisée de BERT [26], qui offre les meilleurs résultats. Nous avons ensuite affiné DistilBERT sur nos données spécifiques pour classifier les descriptions de vulnérabilités.

Collecte de données

Nous avons utilisé les vulnérabilités fournies par le NVD de 2011 jusqu'à avril 2024, que nous avons récupérées au format JSON. Nous avons ensuite conservé uniquement les vulnérabilités disposant de métriques de criticité CVSS. Pour chacune des 8 métriques, nous avons extrait uniquement la description de la vulnérabilité en anglais ainsi que la classe associée à cette métrique, puis nous les avons enregistrées dans des fichiers CSV.

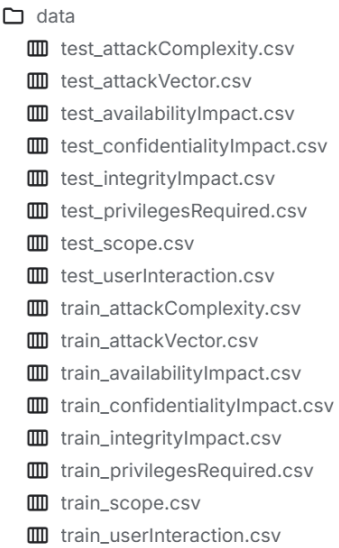


Fig. 4.7 : Dataset

Pour chaque métrique CVSS, nous avons utilisé 80 % des données pour l'entraînement et 20 % pour le test. La table 4.1 présente la distribution du jeu de données pour chaque

métrique.

Catégorie	Entraînement	Test
attackVector		
ADJACENT NETWORK	**2494**	**623**
LOCAL	**28224**	**7056**
NETWORK	**87166**	**21791**
PHYSICAL	**1207**	**302**
attackComplexity		
HIGH	**7329**	**1832**
LOW	**111762**	**27940**
scope		
CHANGED	**21084**	**5271**
UNCHANGED	**98007**	**24501**
privilegesRequired		
HIGH	**9906**	**2477**
LOW	**34874**	**8717**
NONE	**74311**	**18578**
userInteraction		
NONE	**77167**	**19291**
REQUIRED	**41924**	**10481**
availabilityImpact		
HIGH	**68998**	**17249**
LOW	**2677**	**669**
NONE	**47416**	**11854**
confidentialityImpact		
HIGH	**69412**	**17352**
LOW	**24250**	**6063**
NONE	**25429**	**6357**
integrityImpact		
HIGH	**59546**	**14885**
LOW	**22817**	**5705**
NONE	**36728**	**9182**

Tab. 4.1 : La distribution des vulnérabilités selon les catégories du CVSS

Architecture du modèle

- **a) Prétraitement des descriptions**
  But : Transformer les descriptions des vulnérabilités en une forme normalisée adaptée à l'entrée du modèle. Cela inclut la tokenisation et la normalisation.

  Étapes :

  - **Tokenisation :**Diviser le texte en unités significatives (tokens).

  - **Lemmatisation :**réduit les mots à leur forme de base ou racine, facilitant la normalisation et l'analyse linguistique.

- **b) Encodage des descriptions**
  But : Convertir les textes en une représentation numérique que le modèle peut traiter.

  Paramètres :

  - **Tokenizer :** Utilisé pour transformer les textes en séquences de tokens numériques.
  - **max_sequence_length :**Longueur maximale des séquences d'entrée, assurant que toutes les séquences sont de la même longueur pour le traitement uniforme.

- **c) DistilBERT**
  But : Les séquences d'entrée sont traitées dans les deux directions grâce à l'architecture bidirectionnelle de DistilBERT. Cela permet au modèle de capturer les dépendances contextuelles des mots tant dans le passé que dans le futur, améliorant ainsi la compréhension globale du texte.

  Paramètres :

  - **num_hidden_layers(6) :** Ce paramètre détermine combien de couches de traitement sont empilées dans le modèle.
  - **num_attention_heads(12) :** Nombre de têtes d'attention dans chaque couche. Chaque tête d'attention peut se concentrer sur différentes parties du texte, permettant au modèle de capturer divers aspects des relations entre les mots.
  - **hidden_size(768) :** Dimension des vecteurs de représentation à travers les couches du modèle .

- **d) Dropout Layer**
  But : Réduire le risque de sur-apprentissage en désactivant aléatoirement un pourcentage des unités pendant l'entraînement, améliorant ainsi la généralisation du modèle.

  Paramètres :

  - **dropout_rate (0.1) :** Le taux de désactivation des unités, fixé à 10%.

- **c) Dense Layer**
  But : Transformer les sorties du Transformer en une forme adaptée à la classification.

  Paramètres :

  - **units (num_classes) :** Nombre de neurones de sortie correspondant au nombre de classes de métriques CVSS prédites.

– **activation (softmax) :**Fonction d'activation Softmax pour convertir les sorties en probabilités.

- **d) Compilation**
  But : Préparer le modèle pour l'entraînement en définissant l'optimiseur, la fonction de perte et les métriques d'évaluation.

  Configurations :

  – **Optimiseur :** Adam - Choisi pour son efficacité et ses capacités d'apprentissage adaptatif.

  – **Fonction de Perte :**Sparse_categorical_crossentropy - Cette fonction de perte est adaptée aux tâches de classification multi-classes

  – **Métriques :**accuracy et F1-Score - Utilisés pour évaluer la performance du modèle pendant l'entraînement et l'évaluation.

Cette architecture, exploitant DistilBERT [25] et des couches denses avec régularisation, est bien adaptée pour traiter les données séquentielles et prédire avec précision les métriques CVSS [3]. L'utilisation de DistilBERT permet une gestion efficace des données textuelles, tandis que ses capacités capturent le contexte des séquences d'entrée dans les deux directions. La figure 4.8 présente un résumé de cette architecture.

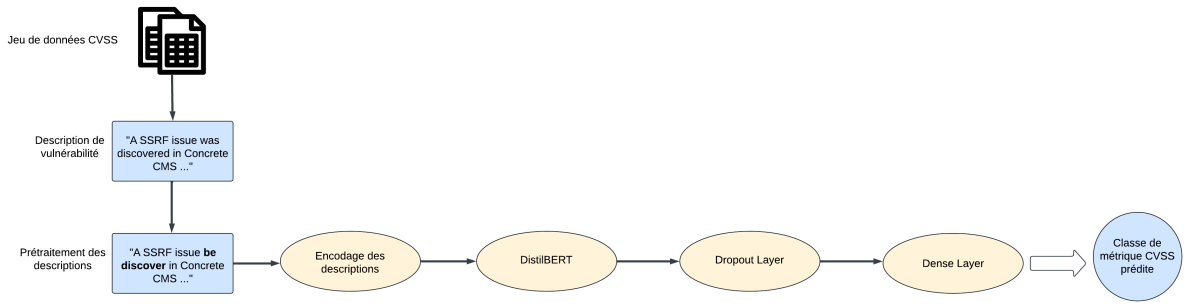


Fig. 4.8 : Architecture du modèle

4.3.3 Extraction de la liste des dépendances

Un SBOM (Software Bill of Materials) est un document détaillé qui répertorie tous les composants logiciels d'une application ou d'un système, y compris les bibliothèques, les dépendances, les licences et les versions utilisées. Il fonctionne comme un inventaire permettant de savoir précisément ce qui constitue un logiciel, ce qui est crucial pour la gestion des vulnérabilités liées aux dépendances logicielles [28].

Le SBOM [28] a plusieurs formats normalisés. Les normes les plus couramment adoptées sont SPDX (Software Product Data Exchange) [28] et CycloneDX [28] comme montré dans la Figure 4.9.

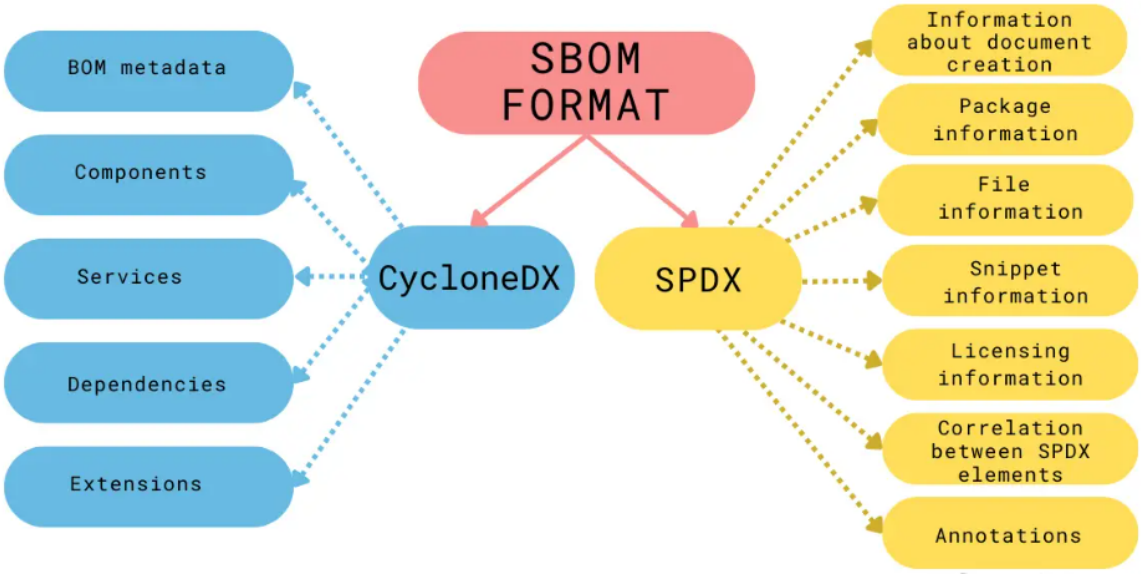


Fig. 4.9 : Les formats normalisés de SBOM [29]

SPDX [28] est un format ouvert et lisible par machine, dont les origines remontent à la Linux Foundation. SPDX est conçu pour faciliter son intégration dans les flux de travail des développeurs et des entreprises afin de soutenir la conformité logicielle et la transparence pour les codes source ouverts et propriétaires.

CycloneDX [28] est également un format open-source pour les SBOM. Il s'agit d'une norme légère de l'OWASP [9], spécialement conçue pour une utilisation dans les contextes de sécurité logicielle et d'analyse des composants de la chaîne d'approvisionnement. CycloneDX [28] permet de communiquer l'inventaire des composants logiciels, des services externes et de leurs relations mutuelles.

Pour choisir entre les deux normes, SPDX [28] est généralement utilisé pour générer des rapports détaillés sur les composants logiciels d'un projet afin de les partager avec des équipes de développement, des responsables de la conformité ou d'autres parties prenantes, tandis que CycloneDX [28] est conçu pour être facilement traité par des outils automatisés, offrant une représentation standardisée des composants logiciels qui peut être intégrée dans des processus de développement et de déploiement automatisés [29].

Nous avons opté pour CycloneDX, car le SBOM généré sera intégré de manière automatisée dans notre processus. Ce SBOM généré, qui contient les informations nécessaires sur les composants logiciels d'une application Java à analyser, servira ensuite à faire le matching entre ces composants logiciels et les composants existants dans notre base de données.

La table 4.2 ci-dessous présente les différentes informations fournies par chacun de deux formats SPDX [28] et CycloneDX [28].

SPDX	CycloneDX
Information sur la création de documents : Elle fournit des informations sur qui, comment et quand le fichier SPDX a été créé, et associe les résultats d'analyse à une version particulière du fichier SPDX.	**BOM metadata :** Elles fournissent des informations sur le fabricant, le fournisseur, les composants cibles, les informations de licence et les outils utilisés pour créer les metadata.
Informations sur le package : Les éléments qui sont des propriétés communes de l'ensemble du package.	**Composants :** Ce champ décrit l'inventaire des composants de première ou de tierce partie.
Informations sur les fichiers : Ce sont des éléments spécifiques aux fichiers pouvant être inclus dans les packages.	**Services** : Il décrit les API externes telles que les exigences d'authentification, les URI de point de terminaison, les traversées de limite de confiance.
Informations extraites : Il s'agit de détails spécifiques à une partie particulière du fichier.	**Dépendances :** Il décrit comment les composants dépendent les uns des autres à travers un graphe de dépendances montrant les relations directes et transitives.
Informations sur les licences : Un moyen de capturer des informations sur les licences et de faire référence à d'autres licences qui ne sont pas fournies dans la liste des licences SPDX.	**Extensions :** Il fournit des points d'extension pour aider les cas d'utilisation et les fonctionnalités futures.
Corrélation entre les éléments SPDX : Il fournit des détails sur la manière dont les documents, fichiers et packages sont liés les uns aux autres.	
Annotations : Informations sur les personnes ayant examiné les fichiers SPDX et la date de révision.	

Tab. 4.2 : Les différentes informations fournies par chacun de deux formats SPDX et CycloneDX [29].

4.3.4 Analyse statique du code source

Notre approche proposée implique une analyse statique du code source de l'application à analyser. Cela comprenait le parcours du code source et l'extraction de sa représentation sous forme d'Abstract Syntax Tree (détaillée dans le deuxième chapitre). L'AST est une structure complexe qui contient des nœuds représentant différentes éléments telles que les classes, les attributs, les méthodes et leurs paramètres respectifs. De plus, l'AST inclut également les interdépendances entre ces classes. En somme, l'AST extrait nous permet de comprendre et de naviguer systématiquement dans l'architecture du code, ce qui nous permet de connaître quels éléments existent dans le code source.

Comme le montre la figure 4.10, nous extrayons l'AST du code source des classes d'entrée en utilisant un outil spécifique, afin d'obtenir finalement l'AST global du code source.

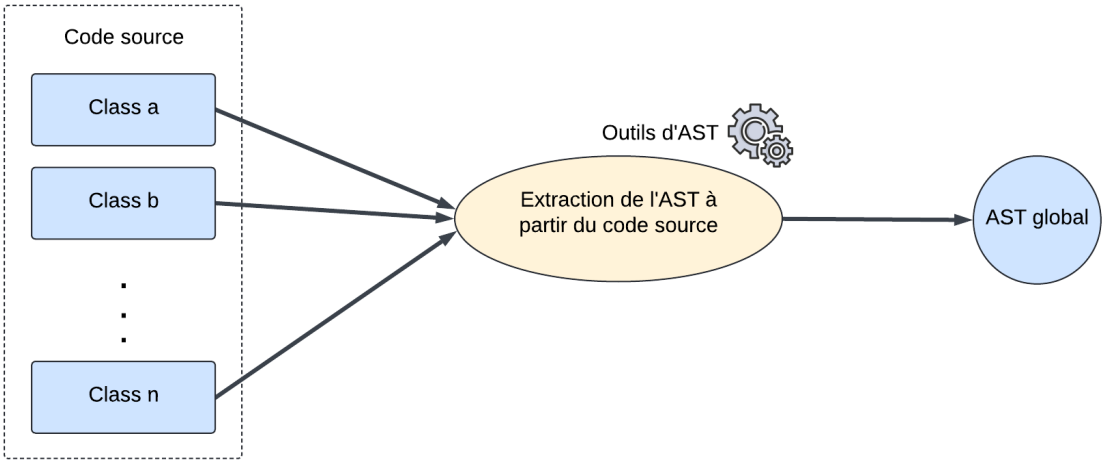


Fig. 4.10 : Processus d'extraction de l'AST

4.3.5 Détection des vulnérabilités

L'algorithme 1 détecte les vulnérabilités dans les composants d'une base de code source en vérifiant d'abord les commits correctifs disponibles. Pour chaque dépendance ou composant du code source, l'algorithme extrait son arbre syntaxique abstrait (AST) et recherche les vulnérabilités connues qui ont un commit correctif associé. En comparant les AST des éléments du composant avec ceux des éléments vulnérables présents dans les commits correctifs, l'algorithme identifie les vulnérabilités corrigées. Si un élément vulnérable est trouvé parmi les éléments de dépendance, la vulnérabilité est signalée comme étant présente dans le composant.

Si la vulnérabilité n'a pas de commit correctif, l'algorithme passe à une vérification alternative en utilisant le Common Platform Enumeration (CPE). Il consulte les vulnérabilités connues pour le CPE du composant et ajoute ces vulnérabilités à la liste des composants vulnérables si elles sont présentes. L'algorithme retourne ensuite la liste complète des composants vulnérables, offrant ainsi une vue globale des failles potentielles

présentes dans les dépendances du projet d'entrée.

Algorithm 1 : DétecterVulnérabilités(CodeSource, BaseDeDonnéesVulnérabilités)

```
1  begin
2      SBOM ← CodeSource.SBOM()
3      composantsVulnérables ← Liste()
4      for chaque composant ∈ SBOM.composants do
5          astComposant ← extraireAST(composant)
6          for chaque vulnérabilité ∈ getAllVulnerabilities() do
7              commitVulnérableTrouvé ← Faux
8              if vulnérabilité.HasFixCommit = Vrai then
9                  fixCommit ← obtenirCommitCorrectif(vulnérabilité)
10                 for chaque élément ∈ astComposant do
11                     astÉlément ← extraireAST(élément)
12                     for chaque élémVulnérable ∈ fixCommit do
13                         astÉlémVulnérable ← extraireAST(élémVulnérable)
14                         if astÉlément = astÉlémVulnérable then
15                             ajouter(composantsVulnérables, (composant, vulnérabilité))
16                             commitVulnérableTrouvé ← Vrai
17                             break
18                     if commitVulnérableTrouvé then
19                         break
20             else
21                 if composant.cpe ≠ NULL then
22                     listeCPEs ← vulnérabilité.getCpes()
23                     if listeCPEs contient composant.cpe then
24                         ajouter(composantsVulnérables, (composant, vulnérabilité))
25     Retourner composantsVulnérables
```

4.3.6 Priorisation des vulnérabilités détectées

Après la détection des dépendances vulnérables, nous devons prioriser ces vulnérabilités afin que les développeurs se concentrent sur les vulnérabilités les plus critiques. Nous avons prédit automatiquement les 8 métriques du système CVSS [3], utilisées pour prioriser les vulnérabilités. Il est maintenant nécessaire de calculer le score global de criticité à partir de ces 8 métriques.

Pour calculer le score global de criticité, nous avons utilisé la formule employée par le système CVSS : Score global = round up(min(Impact + Exploitability, 10)). Cette formule prend en compte les composantes d'impact et d'exploitabilité des vulnérabilités. Le score final est limité à un maximum de 10 pour garantir que le score reflète la sévérité réelle des vulnérabilités, en respectant les standards de calcul établis par le CVSS.

Étant donné que :

$$\text{Exploitability} = 8.22 AV.AC.PR.UI$$

si le Scope est inchangé

$$\text{Impact} = 1 - [(1 - \text{Conf Impact})(1 - \text{Integ Impact})(1 - \text{Avail Impact})]$$

si le Scope est modifié

$$\text{Impact} = 1.08\,[1 - (1 - \text{Conf Impact})(1 - \text{Integ Impact})(1 - \text{Avail Impact})]$$

Avant de calculer le score CVSS, il est essentiel de comprendre les valeurs des métriques utilisées pour l'évaluation, qui seront insérées dans les équations citées précédemment.

Métrique	**Valeurs**
Vecteur d'attaque (AV)	
Réseau (Network)	0.85
Adjacent (Adjacent)	0.62
Local (Local)	0.55
Physique (Physical)	0.2
Complexité d'attaque (AC)	
Faible (Low)	0.77
Élevée (High)	0.44
Privilèges requis (PR)	
Aucun (None)	0.85
Faible (Low)	0.62
Élevé (High)	0.27
Interaction de l'utilisateur (UI)	
Aucun (None)	0.85
Requise (Required)	0.62
Impact sur la confidentialité (Conf Impact)	
Aucun (None)	0
Faible (Low)	0.22
Élevé (High)	0.56
Impact sur l'intégrité (Integ Impact)	
Aucun (None)	0
Faible (Low)	0.22
Élevé (High)	0.56
Impact sur la disponibilité (Avail Impact)	
Aucun (None)	0
Faible (Low)	0.22
Élevé (High)	0.56

Tab. 4.3 : Les valeurs des métriques CVSS

La table 4.3 présente les différentes valeurs attribuées aux métriques du CVSS, notamment les vecteurs d'attaque, la complexité d'attaque, les privilèges requis, l'interaction de l'utilisateur, ainsi que les impacts sur la confidentialité, l'intégrité et la disponibilité. Ces valeurs permettent de calculer le score global de criticité des vulnérabilités.

4.4 Conclusion

Dans ce chapitre, nous avons détaillé la conception du système de contextualisation des dépendances logicielles en mettant en avant les composants clés, en commençant par la construction de la base de données des vulnérabilités et l'utilisation de l'apprentissage automatique pour la prédiction des métriques CVSS, qui sont ensuite utilisées pour prioriser les vulnérabilités détectées. Ensuite, nous avons décrit les techniques d'analyse du code source, en extrayant le SBOM et l'AST du code source, puis en détectant les dépendances vulnérables présentes dans le code et en les priorisant (la plus critique est la plus prioritaire) afin que les développeurs se concentrent sur les vulnérabilités les plus critiques. Dans le chapitre suivant, nous allons entamer le processus de réalisation de l'outil souhaité et effectuer les tests nécessaires de l'outil réalisé.

Chapitre 5

Réalisation et Tests

5.1 Introduction

Après avoir identifié les besoins de notre solution, il est désormais essentiel de sélectionner les technologies à utiliser pour la réalisation de chaque étape de la solution, telles que détaillées dans le chapitre précédent. Dans ce chapitre, nous présenterons les technologies de développement choisies, en justifiant leur utilisation. Nous détaillerons également les tests effectués pour valider les processus mis en œuvre, garantissant ainsi la robustesse et la fiabilité de notre solution.

5.2 Technologies utilisées

5.2.1 FastAPI

FastAPI est un framework web en Python conçu pour créer des APIs rapides et performantes. Il génère automatiquement de la documentation OpenAPI. Lancé en 2018, il est apprécié pour sa simplicité d'utilisation et ses performances élevées. Son objectif est de faciliter la création d'APIs pour une meilleure gestion des données [30].



Fig. 5.1 : Logo FastAPI

5.2.2 MySQL

MySQL est un système de gestion de bases de données relationnelles (SGBDR) open-source. Il organise les données en tables interconnectées et utilise le langage SQL (Structured Query Language) pour permettre aux programmeurs de créer, modifier et extraire des données, ainsi que de gérer l'accès des utilisateurs. MySQL fonctionne en collaboration avec le système d'exploitation pour implémenter et gérer la base de données, assurer l'intégrité des données et faciliter la création de sauvegardes [31].



Fig. 5.2 : Logo MySQL

5.2.3 Java

Java est un langage de programmation de haut niveau, orienté objet et basé sur des classes, conçu pour fonctionner avec un minimum de dépendances d'implémentation. Il permet aux développeurs d'écrire du code une fois et de l'exécuter sur n'importe quelle plateforme compatible avec Java. Le code Java est généralement compilé en bytecode, exécutable sur toute machine virtuelle Java (JVM). La syntaxe de Java s'inspire de C et C++, tout en étant plus simple et sans les fonctionnalités de bas niveau [32].



FIG. 5.3 : Logo Java

Ce langage de programmation a été sélectionné pour l'implémentation de la logique de la solution en raison de sa compatibilité avec divers outils en ligne de commande. Sa diversité de bibliothèques et de frameworks le rend également particulièrement avantageux.

Voici les bibliothèques qui seront utilisées :

- **io :** la bibliothèque io est un ensemble de classes et d'interfaces qui permet de gérer les opérations d'entrée et de sortie. Elle est utilisée pour lire et écrire des données dans différents types de flux de données, tels que les fichiers, les données provenant de réseaux, ou les entrées utilisateur [33].

- **JavaFX :** JavaFX est une bibliothèque logicielle destinée au développement et à la diffusion d'applications de bureau et d'applications web riches, fonctionnant sur une vaste gamme de dispositifs. Elle permet de concevoir des interfaces utilisateur interactives et graphiquement avancées, facilitant ainsi le développement d'applications en Java [34].

- **JavaParser :** La bibliothèque JavaParser génère un arbre de syntaxe abstraite (AST) à partir du code Java. Cette structure AST permet d'interagir avec le code Java de manière programmatique, facilitant ainsi l'analyse et la manipulation du code [35].

5.2.4 Syft

Syft est un outil open-source conçu pour générer automatiquement des SBOMs (définis dans le chapitre de conception) à partir de systèmes de fichiers. Il permet d'analyser les packages logiciels et leurs dépendances, en extrayant les informations essentielles pour chaque dépendance identifiée. Syft prend en charge les deux formats de SBOM, CycloneDX et SPDX [28], et sera utilisé dans notre cas pour l'extraction du SBOM du projet Java au format CycloneDX [36].



Fig. 5.4 : Logo Syft

5.3 Présentations des outils utilisés

5.3.1 Visual Studio Code

Visual Studio Code, souvent appelé VS Code, est un éditeur de code source développé par Microsoft. Il propose des fonctionnalités telles que le débogage, la coloration syntaxique, les extraits de code, le refactoring, et un contrôle de version intégré avec Git. Il offre une prise en charge native de JavaScript, TypeScript et Node.js, et bénéficie d'un vaste écosystème d'extensions pour d'autres langages et environnements d'exécution [37].



Fig. 5.5 : Logo Visual Studio Code

5.3.2 SQL Server

SQL Server de Microsoft est un système de gestion de bases de données (SGBD) basé sur le langage SQL, intégrant notamment un système de gestion de bases de données relationnelles (SGBDR). Développé et commercialisé par Microsoft, il est compatible avec les systèmes d'exploitation Windows et Linux [38].



FIG. 5.6 : Logo SQL Server

5.3.3 Postman

Postman est une plateforme API qui facilite toutes les étapes du cycle de vie des API, aidant les développeurs à concevoir, tester, modifier et documenter les API de façon plus efficace. Elle propose une interface graphique intuitive pour envoyer et visualiser les requêtes et réponses HTTP [39].

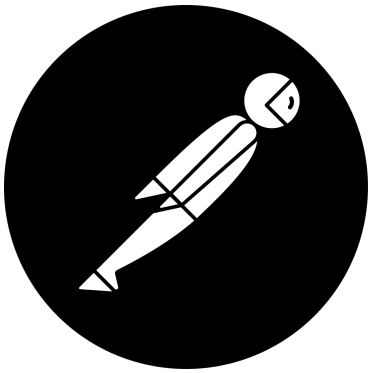


FIG. 5.7 : Logo Postman

5.3.4 Git

Git est un système de contrôle de version distribué qui permet de gérer les versions des fichiers et est largement utilisé pour le développement collaboratif de logiciels. Conçu pour être rapide et fiable, il offre une copie locale complète du dépôt, incluant l'historique, ce qui permet de travailler hors ligne [**Git**].



FIG. 5.8 : Logo Git

5.3.5 Github

GitHub est une plateforme de développement qui permet aux développeurs de créer, stocker, gérer et partager leur code. En utilisant le logiciel Git, elle offre le contrôle de version distribué de Git ainsi que des fonctionnalités supplémentaires telles que le contrôle d'accès, le suivi des bogues et la gestion des tâches [40].

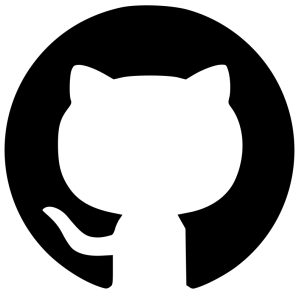


FIG. 5.9 : Logo Github

5.3.6 kaggle

Kaggle est une plateforme en ligne appartenant à Google LLC, dédiée à la science des données. Elle permet aux utilisateurs de créer et d'évaluer des modèles dans un environnement web spécialisé, de collaborer avec des professionnels du domaine et de participer à des défis pour résoudre des problèmes complexes en science des données. Nous avons utilisé Kaggle pour tester et entraîner des modèles de prédiction de la criticité des vulnérabilités à partir de leurs descriptions. [41]



FIG. 5.10 : Logo Kaggle

5.4 Schéma technique de la solution

L'intégration des différentes technologies avec l'architecture conçue, se traduit par le schéma suivant. Cet environnement technique facilite la mise en œuvre de notre système de manière fluide.

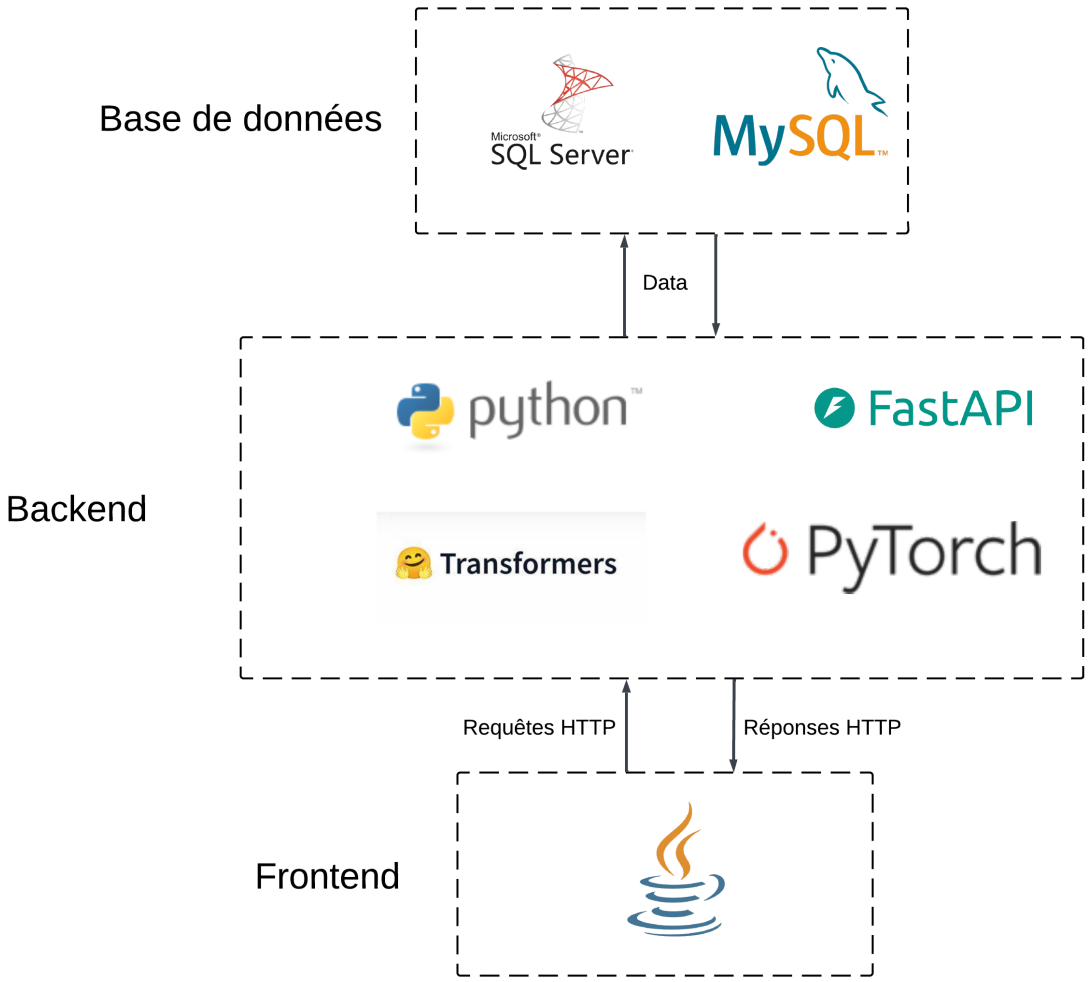


FIG. 5.11 : Schéma technique de la solution

5.5 Implémentation

5.5.1 Intégration d'outils

Syft

Syft [36] sera utilisé pour extraire le SBOM du projet Java à analyser. Il inclut l'analyse du fichier de configuration pom.xml pour extraire l'arbre des dépendances utilisées par le projet à analyser. Pour ce faire, une seule commande sera utilisée, comme suit :

```
syft "cheminProjet" -o cyclonedx-json="nomfichier"
```

- **syft :** C'est le nom de l'outil utilisé.
- **cheminProjet :** Il s'agit d'un paramètre qui représente le chemin du répertoire du projet à analyser.
- **-o :** Ce flag indique à Syft le format de sortie souhaité.

- **cyclonedx-json :**Cela spécifie que le format de la SBOM sera CycloneDX en JSON.
- **nomfichier :** C'est le nom du fichier de sortie où la SBOM sera enregistrée.

Cet outil de ligne de commande doit être installé manuellement, car il n'est pas inclus par défaut avec le système d'exploitation.

Comme indiqué dans le chapitre de conception, le format choisi pour le SBOM généré est CycloneDX, et il sera enregistré dans un fichier JSON.

5.5.2 Implémentation des modèles de transfert learning

Après avoir proposé une conception détaillée de nos modèles de transfert learning destinés à prédire les métriques de criticité CVSS [3] en exploitant les descriptions des vulnérabilités, nous nous concentrons désormais sur la phase d'implémentation de ces modèles. Cette section détaillera le codage, l'entraînement, le finetuning et les tests de ces modèles avant leur déploiement dans notre outil informatique.

Codage des Modèles

- **Choix des Cadres et Bibliothèques**
  Pour l'implémentation du modèle de prédiction des métriques de criticité CVSS, plusieurs bibliothèques et outils ont été sélectionnés :
  - **NLTK (Natural Language Toolkit) :** Utilisé pour le prétraitement du texte, incluant la tokenisation et la lemmatisation.
  - **PyTorch :** Framework utilisé pour la création des jeux de données personnalisés et la gestion des modèles de classification de séquence.
  - **Transformers de Hugging Face :** Utilisé pour la sélection de modèles préentraînés et la gestion des processus d'entraînement.
  - **scikit-learn :** Employé pour la division des jeux de données et l'évaluation des performances des modèles.
  - **NumPy :** Utilisé pour les calculs numériques et la manipulation des données sous forme de tableaux.
- **Prétraitement des Données**
  Le prétraitement est une étape cruciale pour la qualité des données en entrée du modèle. Les étapes réalisées incluent :
  - **Lemmatisation :** Le processus de réduction des mots à leur forme de base, en utilisant les outils de NLTK pour lemmatiser les textes selon leur catégorie grammaticale.
  - **Tokenisation :** Séparation des textes en mots individuels, étape réalisée avec NLTK avant l'encodage des textes.
  - **Encodage des Jeux de Données :** Les textes sont encodés en séquences numériques grâce aux tokenizers fournis par Hugging Face.

- **Architecture du Modèle**
  Nous avons conçu un modèle transfert learning en utilisant DistilBERT :
  - **Tokenisation :** Les données textuelles sont tokenisées pour garantir des séquences d'entrée uniformes pour le modèle.
  - **DistilBERT :** Utilisation de DistilBERT pré-entraîné pour générer des embeddings contextuels pour le texte d'entrée.
  - **Couche Dropout** a pour but de réduire le risque de sur-apprentissage en désactivant aléatoirement un pourcentage des unités pendant l'entraînement, améliorant ainsi la généralisation du modèle.
  - **Couche dense** a pour but de capturer les relations complexes entre les caractéristiques extraites par les couches précédentes du modèle pour transformer les sorties du Transformer en une forme adaptée à la classification.
  - **Couche de Sortie :** Une fonction d'activation softmax est utilisée dans la couche finale pour classifier l'entrée dans les catégories souhaitées.

- **Détails de l'Implémentation**
  Les étapes clés de l'implémentation comprennent :
  - **Encodage des variables catégorielles :** Transformation des variables catégorielles en format numérique.
  - **Division des Données :** Utilisation de `scikit-learn` pour diviser les jeux de données en ensembles d'entraînement et de test.(Voir la Figure 5.12)

```
# Train-test split
train_data, val_data = train_test_split(data, test_size=0.2, random_state=42,stratify=data['attackComplexity'])
train_texts = train_data['description']
train_labels = train_data['attackComplexity']
val_texts = val_data['description']
val_labels = val_data['attackComplexity']
```

Fig. 5.12 : Division des Données

  - **Fine-tuning du Modèle :** Ajustement du modèle pré-entraîné en utilisant le framework `PyTorch` et les outils de `Hugging Face`, avec optimisation des hyperparamètres pour améliorer les performances.

Entraînement du Modèle

- **Données d'entraînement**
  Les jeux de données d'entraînement et de test sont constitués de textes encodés décrivant les vulnérabilités, associés à leurs labels de métriques CVSS prédits. Les données sont chargées à partir de fichiers CSV et préparées par lemmatisation et tokenisation.

- **Procédure d'entraînement**
  La procédure d'entraînement a impliqué les étapes suivantes : (Voir la Figure 5.13)

– **Entraînement du Modèle :** Le modèle est entraîné sur les jeux de données encodés en utilisant les arguments de configuration définis pour les hyperparamètres, tels que la taille des batchs et le taux d'apprentissage.

– **Évaluation des Performances :** Les performances du modèle sont surveillées en temps réel à l'aide des métriques d'exactitude (accuracy) et de F1-score.

```
if __name__ == '__main__':
    main(num_labels=2, classes_names='NONE,REQUIRED',  output_dir='output/userInteraction',
         model='distilbert',train_batch=8, epochs=3,lr=3e-5, lemmatization=True,
         stemming=False, rem_stop_words=False)
```

FIG. 5.13 : Entraînement de DistilBERT

5.6 Utilisation de l'application

Pour utiliser l'outil de manière efficace, voici les étapes à suivre :

1. Lancer l'outil.

2. Sélectionner l'application Java en entrée.

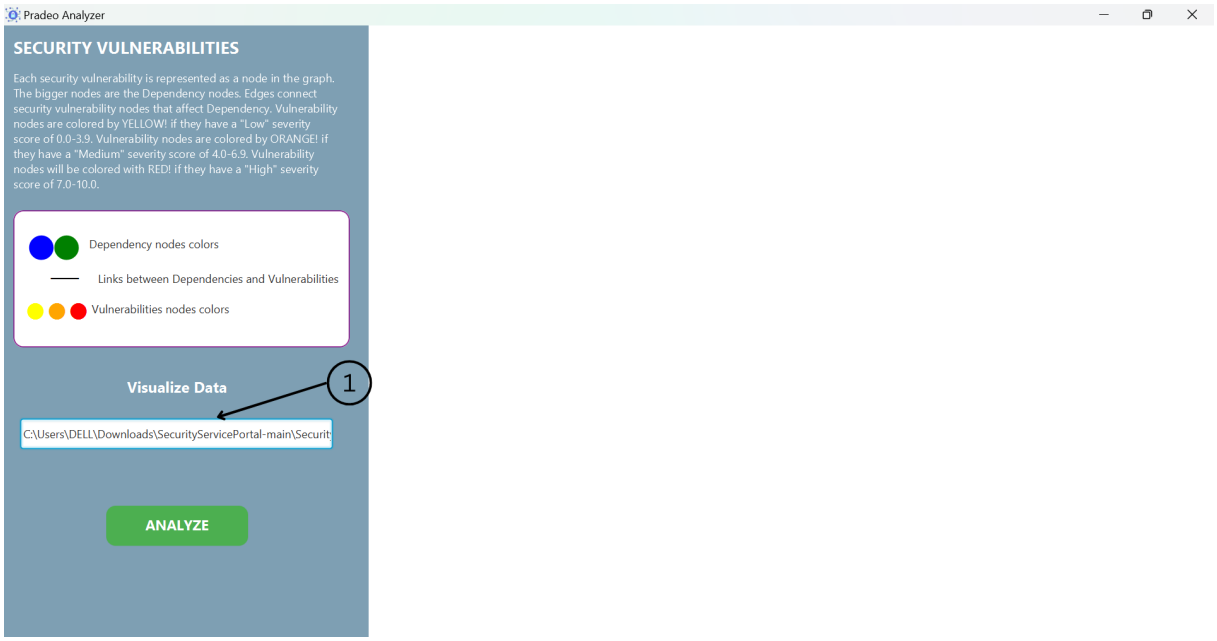


FIG. 5.14 : Sélection de l'application Java en entrée

3. Lancer la contextualisation et attendre le résultat.

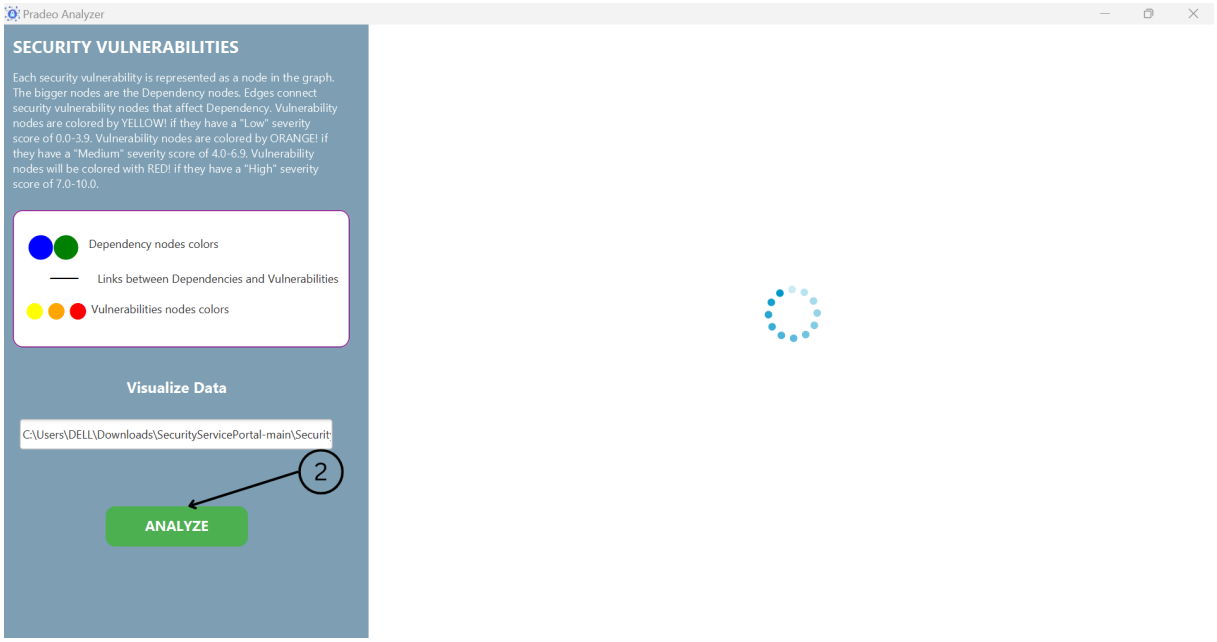


FIG. 5.15 : Lancement de la contextualisation

4. Visualiser le rapport sous forme de graph, avec les vulnérabilités associées aux dépendances du projet en entrée.

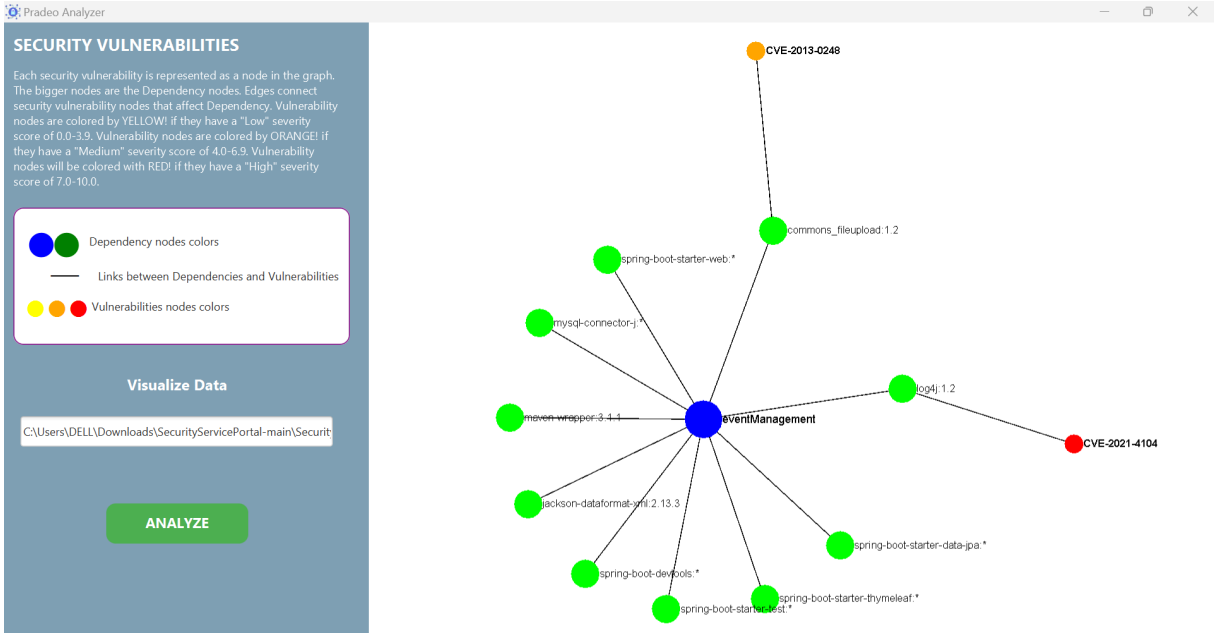


FIG. 5.16 : Visualisation du rapport

5. Fermer l'outil.

5.7 Tests

Les tests jouent un rôle crucial dans le développement logiciel en garantissant que le produit final répond aux exigences et aux attentes définies. Ils permettent de vérifier la fonctionnalité et la fiabilité du logiciel tout en identifiant et en contextualisant les dépendances vulnérables. Un processus de test rigoureux contribue non seulement à assurer la qualité du produit, mais aussi à minimiser les risques et à améliorer la satisfaction des développeurs, qui sont les principaux utilisateurs finaux de notre système.

5.7.1 Approche de test

Il existe plusieurs types de tests pour évaluer les systèmes informatiques. Dans notre cas, nous avons évalué notre système en appliquant les tests suivants :

Tests d'Intégration

Les tests d'intégration vérifient le bon fonctionnement des différentes unités ou modules pour s'assurer qu'ils fonctionnent correctement.

Dans notre cas, nous allons évaluer Syft, qui est exécuté à travers la ligne de commande. Les résultats de l'outil montrent le bon fonctionnement de Syft, comme indiqué dans la figure 5.17.


```
"components": [
    {
        "bom-ref": "pkg:maven/com.fasterxml.jackson.dataformat/jackson-dataformat-xml@2.13.3?
        package-id=b34353e331503df4",
        "type": "library",
        "group": "com.fasterxml.jackson.dataformat",
        "name": "jackson-dataformat-xml",
        "version": "2.13.3",
        "cpe": "cpe:2.3:a:com.fasterxml.jackson.dataformat:jackson-dataformat-xml:2.13.3:*:*:*:*:*:*:*",
        "purl": "pkg:maven/com.fasterxml.jackson.dataformat/jackson-dataformat-xml@2.13.3",
        "properties": [ ···
        ]
    },
    {
        "bom-ref": "pkg:maven/org.apache.maven.wrapper/maven-wrapper@3.1.1?package-id=5995117b42adf045",
        "type": "library",
        "group": "org.apache.maven.wrapper",
        "name": "maven-wrapper",
        "version": "3.1.1",
        "licenses": [ ···
        ],
        "cpe": "cpe:2.3:a:apache:maven-wrapper:3.1.1:*:*:*:*:*:*:*",
        "purl": "pkg:maven/org.apache.maven.wrapper/maven-wrapper@3.1.1",
        "externalReferences": [ ···
        ],
        "properties": [ ···
        ]
    },
```


Fig. 5.17 : Exemple de résultat d'intégration de Syft

Tests de Système

Les tests de système évaluent le système logiciel complet en tant qu'entité intégrée. Ils vérifient que l'ensemble du système fonctionne correctement. Dans notre cas, nous allons évaluer notre système à travers une étude de cas : une application Java dédiée à la gestion des événements. Cela permettra de garantir le bon fonctionnement de notre système en tant qu'entité unique.

D'abord, le processus commence par l'extraction des dépendances utilisées par l'application en utilisant Syft. La sortie de l'outil Syft est montrée dans la figure 5.18.


```
{"components": [
       {   "group": "apache",
           "name": "commons-fileupload",
           "version": "1.2",
           "cpe": "cpe:2.3:a:apache:commons-fileupload:1.2:*:*:*:*:*:*:*",
       },
       {   "group": "com.fasterxml.jackson.dataformat",
           "name": "jackson-dataformat-xml",
           "version": "2.13.3",
           "cpe": "cpe:2.3:a:com.fasterxml.jackson.dataformat:jackson-dataformat-xml:2.13.3:*:*:*:*:*:*:*",
       },
       {   "group": "apache",
           "name": "log4j",
           "version": "1.2",
           "cpe": "cpe:2.3:a:apache:log4j:1.2:*:*:*:*:*:*:*",
       },
       {   "group": "org.apache.maven.wrapper",
           "name": "maven-wrapper",
           "version": "3.1.1",
           "cpe": "cpe:2.3:a:apache:maven-wrapper:3.1.1:*:*:*:*:*:*:*",
       },
       {   "group": "org.springframework.boot",
           "name": "spring-boot-starter-data-jpa",
           "version": "1.14.3",
           "cpe": "cpe:2.3:a:spring-boot-starter-data-jpa:spring-boot-starter-data-jpa:1.14.3:*:*:*:*:*:*:*",
       },
       {   "group": "org.springframework.boot",
           "name": "spring-boot-starter-thymeleaf",
           "version": "5.1",
           "cpe": "cpe:2.3:a:spring-boot-starter-thymeleaf:spring-boot-starter-thymeleaf:5.1:*:*:*:*:*:*:*",
       },
       {   "group": "org.springframework.boot",
           "name": "spring-boot-starter-web",
           "version": "2.1.8",
           "cpe": "cpe:2.3:a:org.springframework.boot:spring-boot-starter-web:2.1.8:*:*:*:*:*:*:*",
       }
   ]

}
```


Fig. 5.18 : Résultat de Syft

En appliquant l'algorithme de détection présenté précédemment, le système trouve que la dépendance log4j :1.2 est vulnérable à CVE-2021-4104 et que commons_fileupload :1.2 est vulnérable à CVE-2013-0248, en utilisant l'approche du matching des vulnérabilités basée sur les noms, car ces deux vulnérabilités n'ont pas de commit de correction.

En revanche, commons_fileupload :1.2 n'est pas vulnérable à CVE-2014-0050, car cette vulnérabilité possède un commit de correction, mais le code vulnérable exploité à partir de l'AST du code supprimé dans le commit de correction n'est pas référencé dans l'AST de l'application. Notre approche ne détecte pas cette vulnérabilité malgré que le composant commons_fileupload :1.2 soit vulnérable à cette vulnérabilité en utilisant le matching basée sur le nom, car elle donne la priorité au matching basée sur le code si un commit de correction existe, comme dans ce cas, pour une meilleure contextualisation.

La figure 5.19 montre les modifications de code liées au fix commit de CVE-2014-0050. Le code vulnérable (le code supprimé) correspond à la méthode getString(String encoding), qui n'est pas référencée dans l'application.

```
src/main/java/org/apache/commons/fileupload/FileItem.java
@@ -20,11 +20,8 @@ public class FileItem {
     /**
      * Gets the contents of the file item as a String, using the specified encoding. This method uses {
@link #get()} to retrieve the contents of the item.
      *
-     * @param encoding The character encoding to use.
-     * @param toCharset The character encoding to use.
+     * @param toCharset The character encoding to use.
      *
      * @return The contents of the item, as a string.
      *
-     * @throws UnsupportedEncodingException if the requested character encoding is not availabl
      * @throws IOException                  eif an I/O error occurs
      */
-    String getString(String encoding) throws UnsupportedEncodingException, IOException;
+    String getString(Charset toCharset) throws IOException;

    /**
     * Tests whether or not a {@code FileItem} instance represents a simple form field.
     */
```

Fig. 5.19 : Modifications de code liées au fix commit de CVE-2014-0050

La figure 5.20 illustre le rapport de contextualisation des dépendances vulnérables de l'application étudiée, en colorant les nœuds des vulnérabilités détectées selon leur criticité relative aux métriques CVSS.

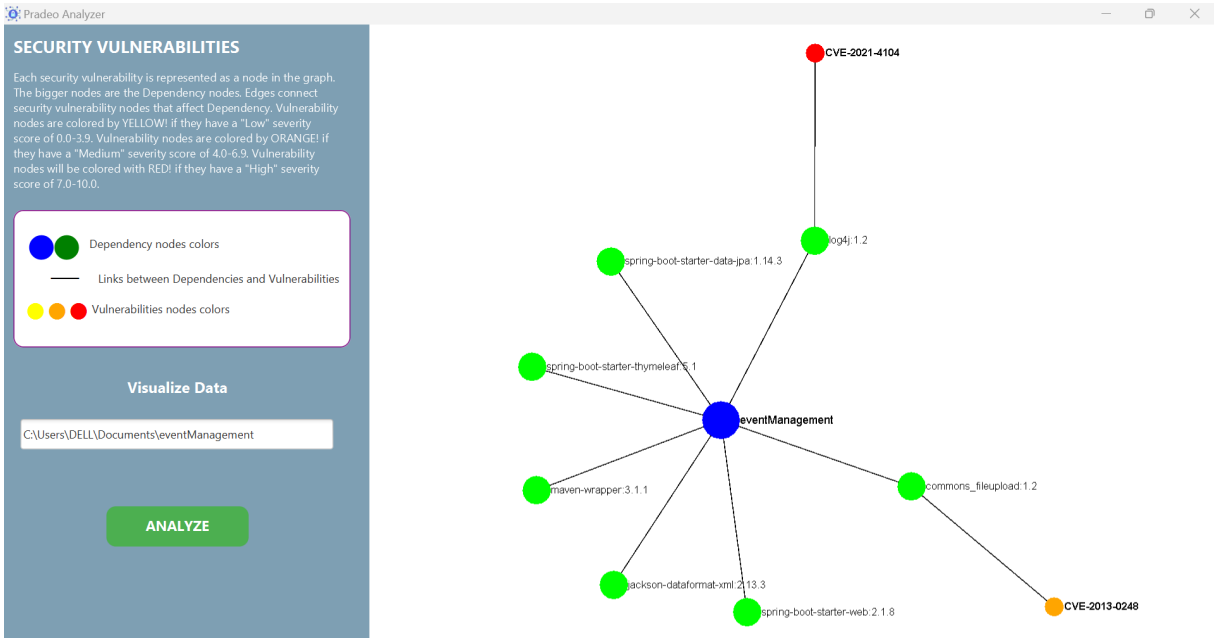


FIG. 5.20 : Rapport de contextualisation des dépendances vulnérables de l'application étudiée

Test et évaluation des modèles de prédiction des métriques CVSS

Pour tester les modèles de prédiction des huit métriques CVSS, nous utilisons les mesures suivantes :

$$\text{Accuracy} = \frac{\text{Nombre de Prédictions Correctes}}{\text{Nombre Total de Prédictions}}$$

$$\text{Précision} = \frac{\text{Vraies Positives (VP)}}{\text{Vraies Positives (VP)} + \text{Fausses Positives (FP)}}$$

$$\text{Rappel} = \frac{\text{Vraies Positives (VP)}}{\text{Vraies Positives (VP)} + \text{Fausses Négatives (FN)}}$$

$$\text{Score F1} = \frac{2\text{PrécisionRappel}}{\text{Précision} + \text{Rappel}}$$

Où :

- **Vraies Positives (VP) :** Nombre de cas correctement identifiés comme positifs.
- **Fausses Positives (FP) :**Nombre de cas incorrectement identifiés comme positifs.
- **Fausses Négatives (FN) :** Nombre de cas incorrectement identifiés comme négatifs.

La table ci-dessous présente les quatre mesures choisies pour évaluer les modèles de prédiction des métriques CVSS :

Champ	Contenu	accuracy	précision	rappel	F1-score
Attack Vector	Indique comment l'attaque est réalisée.	91.92	91.71	91.88	91.76
Attack Complexity	Indique la complexité pour faire l'attaque.	96.29	95.94	95.86	95.9
Privileges Required	Indique les privilèges requis pour être capable de lancer l'attaque.	84.6	84.3	84.35	84.32
Confidentiality Impact	Indique l'impact sur la confidentialité des données si la vulnérabilité a été exploitée.	87.16	87.13	86.94	87.02
Integrity Impact	Indique l'impact sur l'intégrité des données.	87.98	88.03	88.05	87.96
Availability Impact	Indique l'impact sur la disponibilité du système.	89.49	89.3	88.94	89.09
Scope	Indique l'étendue de l'impact de la vulnérabilité. Les valeurs possibles sont : Unchanged (Indique que l'exploitation de la vulnérabilité n'affecte que les ressources directement accessibles ou manipulées par l'application), Changed (Indique que l'exploitation de la vulnérabilité peut entraîner une modification de l'environnement ou des ressources qui sont en dehors de la portée directe de l'application).	96.64	96.6	96.61	96.58
User Interaction	Indique si l'exploitation d'une vulnérabilité nécessite l'interaction de l'utilisateur	93.69	93.67	93.66	93.66

Tab. 5.1 : Test et évaluation des modèles de prédiction des métriques CVSS

5.8 Limitations de la solution

1. Dans notre base de données de vulnérabilités, il existe des vulnérabilités qui n'ont pas de commits correctifs. Nous les détectons donc en appliquant un matching basé sur les noms, ce qui diminue l'efficacité de la contextualisation.

2. Syft n'est pas capable d'extraire la liste des dépendances transitives qui ne sont pas déclarées dans le fichier de configuration pom.xml.

3. Notre outil n'est pas capable de capturer le code vulnérable qui existe dans des méthodes et des classes non identifiées dans les modifications de code des fix commits.

5.9 Conclusion

Ce chapitre a fourni une vue détaillée des étapes de réalisation et de tests de notre système, proposé dans le chapitre de conception.

La réalisation nous a conduits à mettre en place un processus de détection des dépendances vulnérables et de priorisation des vulnérabilités détectées. Elle nous a également conduits à une meilleure contextualisation des dépendances vulnérables.

Grâce à l'utilisation de plusieurs types de tests, nous avons pu valider l'efficacité et le bon fonctionnement de notre système de contextualisation des dépendances logicielles vulnérables.

Conclusion générale et prespectives

La contextualisation des dépendances vulnérables est un sujet de recherche actif. De nombreux travaux de recherche visent à contribuer à l'état de l'art sur ce sujet, chacun ayant un but précis pour améliorer le processus de contextualisation.

Le processus de contextualisation des dépendances vulnérables au sein d'une application consiste d'abord à détecter les dépendances vulnérables dans le contexte de l'application analysée, puis à prioriser les vulnérabilités détectées pour aider les développeurs à se concentrer sur les vulnérabilités les plus critiques.

Dans ce projet de fin d'étude, nous avons d'abord commencé par présenter un état de l'art sur le sujet. Dans cette partie, nous avons exploré les concepts clés liés aux dépendances logicielles et aux vulnérabilités associées à ces dépendances. Nous avons également examiné les approches d'identification des dépendances vulnérables ainsi que les approches de prédiction de la criticité des vulnérabilités en se basant sur l'apprentissage automatique.

Ensuite, nous avons schématisé les spécifications de notre système à travers des diagrammes UML, ce qui nous a permis de définir l'étape de conception et de bien comprendre les exigences de l'outil à réaliser.

Enfin, nous avons décrit l'étape de réalisation de l'outil proposé, Pradeo Analyzer, lors de la phase de conception, suivie par une série de tests visant à valider le bon fonctionnement de l'outil développé.

Pour conclure, nous souhaitons suggérer quelques pistes et perspectives intéressantes à explorer à l'avenir :

- Proposer une nouvelle approche basée sur l'intelligence artificielle pour trouver les commits de correction des vulnérabilités.
- Proposer d'autres techniques d'analyse prenant en compte l'aspect sémantique du code source.
- Intégrer Maven pour identifier la liste des dépendances logicielles transitives qui ne sont pas détectées par Syft.

.

Bibliographie

[1] Ivan Pashchenko et al. “Vuln4Real : A Methodology for Counting Actually Vulnerable Dependencies”. In : *IEEE Transactions on Software Engineering* 48.5 (2022), p. 1592-1609. doi : 10.1109/TSE.2020.3025443.

[5] Serena Elisa Ponta, Henrik Plate et Antonino Sabetta. “Beyond Metadata : Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software”. In : *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2018, p. 449-460. doi : 10.1109/ICSME.2018.00054.

[6] Keehyung Kim et Byung-Ro Moon. “Malware detection based on dependency graph using hybrid genetic algorithm”. In : *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation*. GECCO ’10. Portland, Oregon, USA : Association for Computing Machinery, 2010, p. 1211-1218. isbn : 9781450300728. doi : 10.1145/1830483.1830703.

[7] Ernesto Rosario Russo et al. “Summarizing vulnerabilities’ descriptions to support experts during vulnerability assessment activities”. In : *J. Syst. Softw.* 156.C (oct. 2019), p. 84-99. issn : 0164-1212. doi : 10.1016/j.jss.2019.06.001.

[8] Nasif Imtiaz, Seaver Thorn et Laurie Williams. “A comparative study of vulnerability reporting by software composition analysis tools”. In : *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ESEM ’21. Bari, Italy : Association for Computing Machinery, 2021. isbn : 9781450386654. doi : 10.1145/3475716.3475769.

[15] Yulun Wu et al. “Understanding the Threats of Upstream Vulnerabilities to Downstream Projects in the Maven Ecosystem”. In : *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, p. 1046-1058. doi : 10.1109/ICSE48619.2023.00095.

[16] Mircea Cadariu et al. “Tracking known security vulnerabilities in proprietary software systems”. In : *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)* (2015), p. 516-519.

[17] Serena Ponta, Henrik Plate et Antonino Sabetta. *Beyond Metadata : Code-centric and Usage-based Analysis of Known Vulnerabilities in Open-source Software*. Juin 2018.

[18] Sultan S. Alqahtani, Ellis E. Eghan et Juergen Rilling. “Tracing known security vulnerabilities in software repositories – A Semantic Web enabled modeling approach”. In : *Science of Computer Programming* 121 (2016). Special Issue on

Knowledge-based Software Engineering, p. 153-175. ISSN : 0167-6423. DOI : https://doi.org/10.1016/j.scico.2016.01.005.

[20] Joana Cabral COSTA et al. "Predicting CVSS Metric via Description Interpretation". In : *IEEE Access* 10 (2022), p. 59125-59134. DOI : 10.1109/ACCESS.2022.3179692.

[21] Atefeh KHAZAEI, Mohammad GHASEMZADEH et Vali DERHAMI. "An automatic method for CVSS score prediction using vulnerabilities description". In : *Journal of Intelligent Fuzzy Systems* 30 (août 2015), p. 89-96. DOI : 10.3233/IFS-151733.

[22] Clément ELBAZ, Louis RILLING et Christine MORIN. "Fighting N-day vulnerabilities with automated CVSS vector prediction at disclosure". In : *Proceedings of the 15th International Conference on Availability, Reliability and Security*. ARES '20. Virtual Event, Ireland : Association for Computing Machinery, 2020. ISBN : 9781450388337. DOI : 10.1145/3407023.3407038.

[23] Ion BABALAU et al. "Severity Prediction of Software Vulnerabilities based on their Text Description". In : *2021 23rd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*. 2021, p. 171-177. DOI : 10.1109/SYNASC54541.2021.00037.

[24] Zhuobing HAN et al. "Learning to Predict Severity of Software Vulnerability Using Only Vulnerability Description". In : *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2017, p. 125-136. DOI : 10.1109/ICSME.2017.52.

[28] Sabato NOCERA et al. "Software Bill of Materials Adoption : A Mining Study from GitHub". In : *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2023, p. 39-49. DOI : 10.1109/ICSME58846.2023.00016.

Webographie

[2] *Mitre cve database*. https ://cve.mitre.org/.

[3] *FIRST*. https ://www.first.org/cvss/.

[4] *National vulnerability database*. https ://nvd.nist.gov/vuln.

[9] *OWASP Dependency Check*. https ://github.com/jeremylong/DependencyCheck.

[10] *Trivy*. https ://aquasecurity.github.io/trivy/v0.17.2/.

[11] *Docker*. https ://docs.docker.com/get-started/overview/.

[12] *Eclipse Steady*. https ://github.com/eclipse/steady.

[13] *Grype*. https ://github.com/anchore/grype.

[14] *Black Duck*. https ://www.synopsys.com/software-integrity/software-composition-analysis-tools/black-duck-sca.html.

[19] *AST*. https ://en.wikipedia.org/wiki/Abstract$_s yntax_t ree$.

[25] *Distilbert*. https ://huggingface.co/docs/transformers/model$_d oc/distilbert$.

[26] *BERT*. https ://huggingface.co/docs/transformers/model$_d oc/bert$.

[27] *Scrum*. https ://www.scrum.org/learning-series/what-is-scrum/.

[29] *SPDX vs CycloneDX*. https ://www.erp-information.com/spdx-vs-cyclonedx ?expand$_a rticle$ = 1.

[30] *FastAPI*. https ://en.wikipedia.org/wiki/FastAPI.

[31] *MySQL*. https ://en.wikipedia.org/wiki/MySQL.

[32] *Java*. https ://en.wikipedia.org/wiki/Java$_(programming_l anguage)$.

[33] *io*. https ://thecodersbreakfast.net/index.php ?post/2012/01/15/java-io-explique-simplement.

[34] *JavaFX*. https ://en.wikipedia.org/wiki/JavaFX.

[35] *javaparser*. https ://javaparser.org/.

[36] *Syft*. https ://github.com/anchore/syft.

[37] *Visual Studio Code*. https ://en.wikipedia.org/wiki/Visual$_S tudio_C ode$.

[38] *Microsoft SQL Server*. https ://fr.wikipedia.org/wiki/Microsoft$_S QL_S erver$.

[39] *Postman*. https ://www.postman.com/product/what-is-postman/.

[40] *GitHub*. https ://en.wikipedia.org/wiki/GitHub.

[41] *Kaggle*. https ://en.wikipedia.org/wiki/Kaggle.

Annexes

Annexe A

Contenu d'une vulnérabilité récupérée de la NVD

La Figure A.1 illustre le contenu d'une vulnérabilité récupérée de la base de données NVD, cette vulnérabilité dispose de l'identifiant CVE-2022-42187. Cette vulnérabilité comprend des informations essentielles telles que sa date de publication, la source qui l'a signalée, ainsi que son statut actuel.

Les informations fournies incluent aussi une description générale de la faille. La vulnérabilité est également évaluée selon des métriques standardisées qui mesurent son impact et sa sévérité (Metriques CVSS [3]), offrant une vue d'ensemble de son risque potentiel.

Les configurations indiquent les systèmes ou les versions précises du logiciel qui sont affectés par cette faille. Enfin, des références externes fournissent des liens vers des ressources supplémentaires et des discussions sur la vulnérabilité, permettant aux chercheurs et aux professionnels de la sécurité de se tenir informés des détails et des évolutions associées à cette vulnérabilité. Ce format d'information aide à comprendre la portée et les implications de la vulnérabilité, facilitant ainsi les processus d'évaluation et de gestion de la sécurité.

```
{
    "id": "CVE-2022-42187",
    "sourceIdentifier": "cve@mitre.org",
    "published": "2022-11-17T04:15:10.950",
    "lastModified": "2023-10-18T16:05:51.957",
    "vulnStatus": "Analyzed",
    "descriptions": [
        {
            "lang": "en",
            "value": "Hustoj 22.09.22 has a XSS Vulnerability in /admin/problem_judge.php."
        }
    ],
    "metrics": {
        "cvssMetricV30": [
            {
                "source": "nvd@nist.gov",
                "type": "Primary",
                "cvssData": {
                    "version": "3.0",
                    "vectorString": "CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N",
                    "attackVector": "NETWORK",
                    "attackComplexity": "LOW",
                    "privilegesRequired": "NONE",
                    "userInteraction": "REQUIRED",
                    "scope": "CHANGED",
                    "confidentialityImpact": "LOW",
                    "integrityImpact": "LOW",
                    "availabilityImpact": "NONE",
                    "baseScore": 6.1,
                    "baseSeverity": "MEDIUM"
                }
            }
        ]
    },
    "configurations": [
        {
            "nodes": [
                {
                    "operator": "OR",
                    "negate": false,
                    "cpeMatch": [
                        {
                            "vulnerable": true,
                            "criteria": "cpe:2.3:a:hustoj:hustoj:22.09.22:*:*:*:*:*:*:*",
                            "matchCriteriaId": "7F9D317C-8136-4514-8E07-250E38E47B48"
                        }
                    ]
                }
            ]
        }
    ],
    "references": [
        {
            "url": "https://github.com/zhblue/hustoj/issues/866",
            "source": "cve@mitre.org",
            "tags": [
                "Exploit",
                "Issue Tracking",
                "Third Party Advisory"
            ]
        }
    ]
}
```


FIG. A.1 : Contenu d'une vulnérabilité récupérée de la NVD